

# Automated Traffic Violation Detection System

## System Design Overview

### Introduction

The system described in this document is an end-to-end pipeline that converts raw traffic-camera footage into structured, evidence-backed violation records suitable for transmission to a concerned authority. The pipeline is organized as a sequence of cooperating stages: image preprocessing, vehicle detection and tracking, a set of per-vehicle violation classifiers (helmet, seatbelt, triple-riding, speed, stop-line, red-light, illegal parking, and wrong-side driving), a temporary evidence database keyed by vehicle identity, a conditional second preprocessing pass restricted to flagged vehicles, and a license-plate recognition stage whose output is merged with the violation record before being emailed to the relevant authority or written to a permanent store. A separate, network-level layer correlates evidence for a flagged vehicle across multiple cameras. Each stage below is described at a level sufficient to understand its role and its key design decisions; the accompanying technical documentation files listed for each stage contain the full mathematical formulation, configuration parameters, and reference implementation.

## 1 Image Acquisition and Preprocessing

The pipeline begins with a raw frame captured by a CCTV camera at an intersection, with no assumption made about illumination, weather, or motion-blur conditions. Rather than applying a fixed, always-on enhancement filter, the system first computes a **cumulative darkness score** from four normalized image statistics — histogram spread, percentage of dark pixels, mean brightness, and entropy — each weighted by an empirically tuned coefficient. Only frames whose score meets or exceeds a calibrated threshold are routed through **Zero-DCE++**, a lightweight, unsupervised low-light enhancement network chosen for its robustness across arbitrary lighting and its suitability for near-real-time inference at the edge. Frames that pass through enhancement are subsequently denoised with a **bilateral filter**, which is applied *after* enhancement specifically because curve-based brightening amplifies sensor noise that was previously masked in dark regions. Both the enhanced and the already-well-lit paths converge at a single, unconditional deblurring step using **Wiener deconvolution**, since motion blur from moving vehicles is a function of vehicle and camera dynamics rather than ambient light, and is therefore corrected on every frame regardless of the darkness decision.

A second, independent sub-pipeline is triggered only after a violation has already been flagged. Once a vehicle's bounding box localizes its number-plate region, the cropped region is passed through **LPDGAN**, a conditional generative adversarial network trained with a combined adversarial and pixel-reconstruction (L1) loss, that performs a second, targeted restoration pass tolerating residual blur or low light that the frame-level correction could not fully resolve. This keeps the computationally expensive GAN-based restoration off the real-time critical path, since it is invoked only for the comparatively rare subset of vehicles already classified as violators (*see [image\\_preprocessing\\_pipeline.pdf](#) for full detail*).

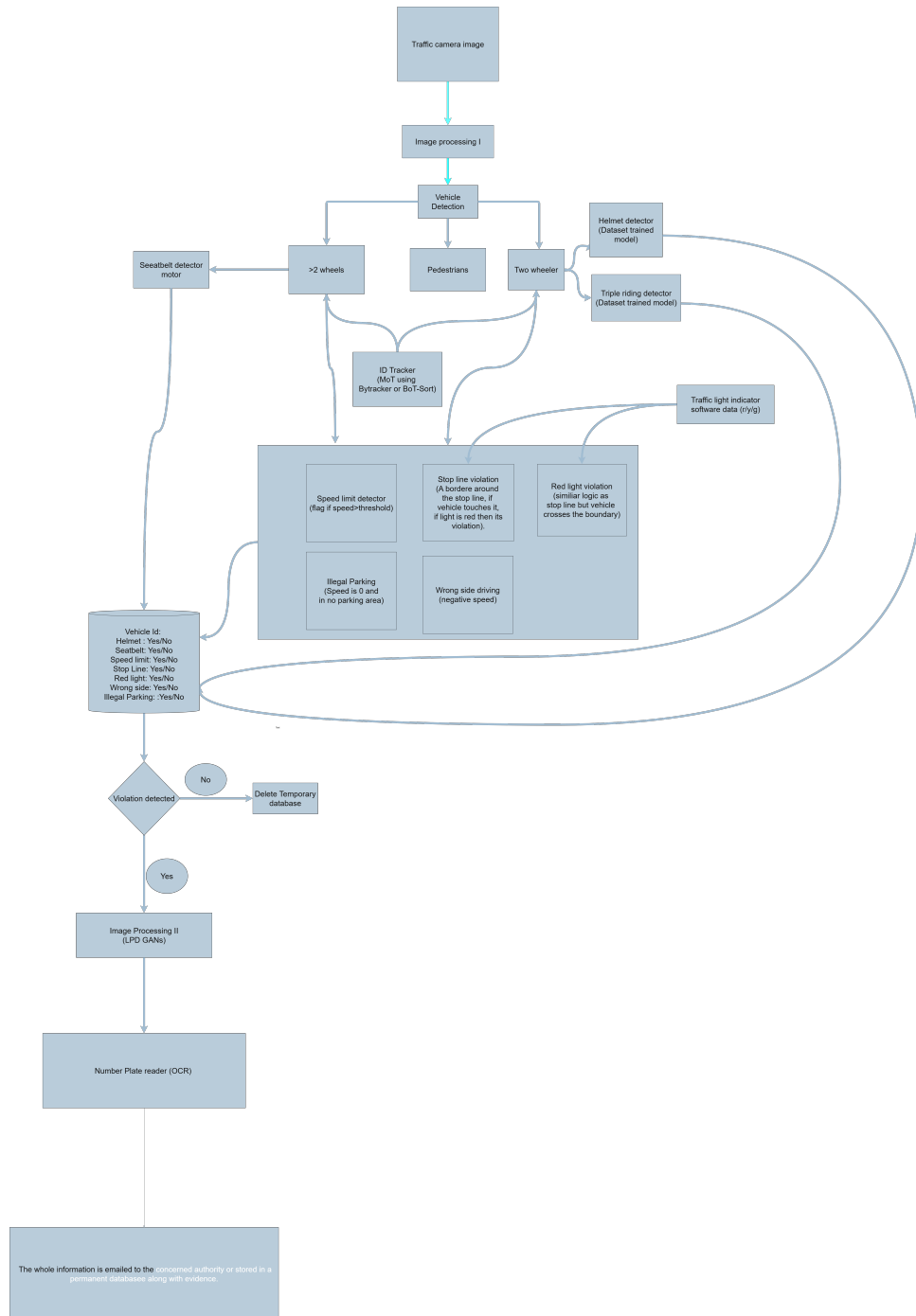


Figure 1: System Architecture.

## 2 Vehicle Detection and Multi-Object Tracking

Object localization is performed by **YOLO26**, queried once per frame to return, for every detected object, a bounding box, a COCO class label, and a confidence score; the nano-scale checkpoint is used for edge-constrained deployment. Detections are linked across frames using **BoT-SORT**, which extends the motion-and-IoU-based ByteTrack with two additional mechanisms suited to a non-static camera and visually similar nearby objects: **Global Motion Compensation**, which estimates the camera’s own inter-frame motion from sparse keypoints and warps each track’s predicted state before matching, preventing camera shake from being misread as object motion; and an optional **Re-Identification** embedding, fused into the cost matrix to reduce identity switches between similar, closely-passing objects. Frame-to-track assignment is solved as a linear bipartite matching problem (the Hungarian algorithm) over a cost combining spatial and, where enabled, appearance distance, with unmatched detections seeding new tracks and unmatched tracks retained as lost for a configurable buffer before termination.

Detected vehicles are split downstream into three categories — pedestrians, two-wheelers, and vehicles with more than two wheels — each routed to the violation logic relevant to that class. Rather than passing every frame of a vehicle’s track to the downstream classifiers, a **crop-extraction** stage reduces each track to exactly one representative image: occupants are associated to a vehicle using **Intersection over the person box’s own Area (IoA)** rather than IoU, since the question being asked is what fraction of a person’s body overlaps the vehicle rather than how similarly sized the two boxes are; for cars, association is restricted to an upper-body zone of the vehicle box, since the seatbelt signal is confined to that region in any case. Across a track’s lifetime, the single best frame is retained using rider count as the primary key and the area of the smallest enclosing box (a clarity proxy) only as a tie-breaker, since under-counting riders could cause a genuine triple-riding violation to be missed. The track is emitted once the tracker reports no detection for it beyond a fixed timeout, guaranteeing exactly one crop per physical vehicle appearance (*see [multi\\_vehicle\\_seatbelt\\_helmet\\_triple\\_detector.pdf](#) for full detail*).

## 3 Per-Vehicle Compliance Classifiers

Three classifiers operate on the crops emitted above. The **helmet detector** and **seatbelt detector** are dataset-trained models evaluating, respectively, two-wheeler riders and car occupants for the presence of a helmet or an engaged seatbelt strap, trained on purpose-built public datasets. The **triple-riding detector** requires no separate model at all: because the crop-extraction stage already associates occupant boxes to a motorcycle via the IoA criterion, a triple-riding violation is deterministically flagged whenever the retained occupant count for the best frame is three or more, avoiding the overhead of an additional dense-prediction model (*see [multi\\_vehicle\\_seatbelt\\_helmet\\_triple\\_detector.pdf](#) for full detail*).

## 4 Speed Estimation

Vehicle speed is computed from the same tracked bounding boxes via a five-stage pipeline: extracting each vehicle’s ground-contact reference point (the bottom-center of its box, the closest 2D approximation to its true road-contact location); projecting that pixel coordinate into real-world metric coordinates via a **homography matrix** calibrated once per camera from four known point correspondences; **plausibility gating**, which rejects any displacement implying a physically impossible speed and clears the affected track’s history entirely rather than discarding only the offending point, since a corrupted track should not silently resume averaging after a discontinuity; and finally computing the windowed Euclidean displacement over a sliding window of history, using elapsed time derived from the video’s native frame rate, and smoothing the

result with an **Exponential Moving Average** to suppress the frame-to-frame jitter that persists even at constant true speed due to bounding-box noise at the window’s endpoints. The **speed limit detector** simply flags a vehicle whose smoothed speed exceeds a configured threshold (*see [speed\\_calculation\\_4\\_final.pdf](#) for full detail*).

## 5 Stop-Line and Red-Light Violation Detection

This stage divides the frame into a Normal Zone before the stop line and a Detection Zone beyond it, using a half-plane test against the stop-line geometry. Critically, the current **traffic signal color is not inferred from the image** but is supplied directly by the traffic control system, which eliminates errors from glare, occlusion, adverse weather, or poor camera angle as a source of signal-misreading. If the signal is not red, every vehicle is immediately classified as no anomaly. If the signal is red and a vehicle’s centroid lies in the Detection Zone, an anomaly is confirmed, and the vehicle’s already-computed speed disambiguates the violation type: a stationary vehicle past the line is a **stop-line violation**, while a still-moving vehicle past the line is an active **red-light violation** (*see [red\\_light\\_and\\_stop\\_line\\_violation.pdf](#) for full detail*).

## 6 Illegal Parking and Wrong-Side Driving Detection

This module branches on a single test: whether a tracked vehicle’s speed is zero. A stationary vehicle is checked against pre-segmented **no-parking zones**; a per-vehicle dwell timer accumulates the time spent at zero speed inside such a zone, and the vehicle is flagged as **illegally parked** only once this timer exceeds a *class-dependent* grace period (nominally 10 seconds for cars versus 60 seconds for buses), which absorbs brief, legitimate halts such as passenger drop-off. A moving vehicle is instead checked against a pre-defined directional sign convention for that road segment; a mismatch between the sign of its lateral position-times-speed product and the zone’s defined convention indicates **wrong-side driving**. Both checks reuse the same enhanced frame and zone segmentation already computed upstream, requiring no additional model inference (*see [illegal\\_parking\\_wrong\\_side\\_driving.pdf](#) for full detail*).

## 7 Temporary Evidence Database and Violation Gate

Every per-vehicle outcome — vehicle ID, and a Yes/No flag for each of helmet, seatbelt, speed limit, stop line, red light, wrong side, and illegal parking — is written to a temporary, per-vehicle record. A single gate, “*Violation detected*,” then checks whether any flag in that record is positive. If none are, the temporary record is deleted immediately, ensuring that compliant vehicles leave no trace in the system and that storage grows only with genuine violations. If at least one flag is positive, the vehicle proceeds to the final stages below.

## 8 Conditional Second-Pass Imaging and Plate Recognition

Only vehicles that clear the violation gate are passed through **Image Processing II**, which invokes the LPDGAN-based plate restoration sub-pipeline described in Section 1, cropping and repairing the number-plate region before it reaches **Optical Character Recognition (OCR)**. The OCR output is accepted only above a defined confidence threshold and with the expected character length for a standard plate format, after which it is merged back into the vehicle’s violation record (*see [image\\_preprocessing\\_pipeline.pdf](#) for full detail*).

## 9 Cross-Camera Evidence Correlation

A network-level layer sits above the single-camera pipeline described in the preceding sections. Each camera continuously writes its local findings — including vehicles with no violations — into a **shared state** store; an entry is deleted if it carries no violation flag, and retained otherwise. The physical camera network is modeled as a **topology graph**, with edge weights representing a learned mean and variance of historical transit time between camera pairs (rather than a naive distance-over-speed-limit estimate), since real transit time depends on the number of intersections and signals along a route as much as on raw distance. For a flagged vehicle, this graph is used to compute, for each reachable downstream camera, a **time search space** — an arrival window centered on the historical mean and widened by its standard deviation — rather than a single predicted timestamp, since a vehicle may be delayed by a signal or, if originally flagged for speeding, may arrive earlier than a naive estimate would suggest. When a downstream camera’s clock falls within this window, the system attempts plate-based confirmation via OCR, falling back to appearance-based matching when OCR confidence or extracted plate length is insufficient, before merging the new sighting into the vehicle’s case file (*see [evidence\\_extractor\\_2.pdf](#) for full detail*).

## 10 Final Reporting

Once a vehicle’s record contains a confirmed violation, an extracted plate number, and any cross-camera evidence, the complete record — together with supporting image evidence — is emailed to the concerned authority or written to a permanent database, closing out the pipeline for that vehicle.

---

*Each section above summarizes the corresponding stage at a high level. The referenced technical documentation files contain the full mathematical formulation, tunable parameters, and reference code for that stage and should be consulted for implementation detail.*

# Image Preprocessing Pipeline

## Technical Documentation & Logic Specification

*Adaptive, quality-aware enhancement of CCTV frames prior to anomaly detection, and a dedicated restoration sub-pipeline for license-plate recognition once an anomaly (violation) is flagged.*

### 1 Overview

The Image Preprocessing Pipeline conditions raw CCTV frames before they are handed to the downstream anomaly (traffic-violation) detector, and subsequently restores a cropped license-plate region before Optical Character Recognition (OCR). The pipeline is split into two cooperating stages, illustrated in Figure 1 and 2:

- (i) **Pre-Anomaly-Detection Stage** – a *quality-gated* enhancement branch that measures how dark an incoming frame is, conditionally brightens and denoises it, and finally deblurs it before it is passed to a YOLO-based detector.
- (ii) **Post-Anomaly-Detection Stage** – once a violation has been detected and a vehicle's number-plate region is cropped, a GAN-based restoration network (LPDGAN) repairs blur/low-light artefacts specifically in that region before OCR is run.

Unlike a fixed, always-on enhancement filter, the pre-detection stage uses an **image-quality score** to decide *whether* aggressive low-light enhancement (Zero-DCE++) is even necessary – avoiding unnecessary artefacts on frames that are already well lit. Every frame, regardless of this decision, is still deblurred via Wiener deconvolution immediately before detection, since motion blur from moving vehicles is independent of ambient illumination.

## 2 Stage 1: Pre-Anomaly-Detection Pipeline

### 2.1 Pipeline Diagram

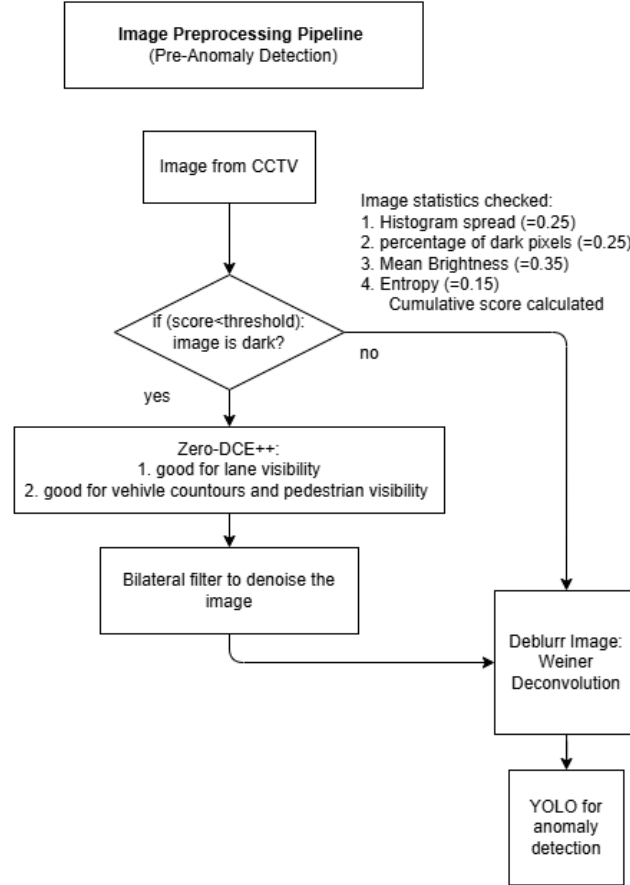


Figure 1: Pre-anomaly Image Processing

## 2.2 Stage Description

### 2.2.1 Image Acquisition

The pipeline takes the raw frame  $I_{raw}$  captured from a CCTV camera at the intersection as input. No assumption is made about illumination, weather, or motion-blur conditions at this point – all such degradations are screened for in the next step.

### 2.2.2 Image Quality Scoring (Darkness Gate)

Rather than always invoking a low-light enhancement network – which can introduce artefacts and unnecessary latency on frames that are already well exposed – the pipeline first computes a **cumulative darkness score**  $Q$  from four normalized statistics of  $I_{raw}$ , each weighted by an empirically tuned coefficient:

$$Q(I_{raw}) = w_1 \cdot (1 - H_{spread}) + w_2 \cdot P_{dark} + w_3 \cdot B_{mean} + w_4 \cdot E_{norm} \quad (1)$$

with weights

$$w_1 = 0.25, \quad w_2 = 0.25, \quad w_3 = 0.35, \quad w_4 = 0.15, \quad \sum_i w_i = 1.$$

Each term is defined as follows, where  $I_{raw}$  is first converted to grayscale with  $L$  intensity levels (typically  $L = 256$ ) and  $N$  is the total pixel count:

**Histogram spread  $H_{spread}$ :** a normalized measure of how widely pixel intensities are distributed across the dynamic range. Letting  $h(i)$  denote the (normalized) intensity histogram,

$$H_{spread} = \frac{\sigma_I}{L-1}, \quad \sigma_I = \sqrt{\sum_{i=0}^{L-1} h(i) (i - \mu_I)^2}, \quad \mu_I = \sum_{i=0}^{L-1} i h(i). \quad (2)$$

A low spread (pixels clustered in a narrow band) is characteristic of flat, dark frames.

**Percentage of dark pixels  $P_{dark}$ :** fraction of pixels below an intensity threshold  $\tau_{dark}$  (e.g.  $\tau_{dark} = 50$  on a 0–255 scale),

$$P_{dark} = \frac{1}{N} \sum_{p \in I_{raw}} \mathbb{I}[I_{raw}(p) < \tau_{dark}]. \quad (3)$$

**Mean brightness  $B_{mean}$ :** normalized average intensity,

$$B_{mean} = 1 - \frac{1}{N(L-1)} \sum_{p \in I_{raw}} I_{raw}(p) \quad (4)$$

(inverted so that a *darker* image yields a *higher* contribution, consistent with the other terms).

**Normalized entropy  $E_{norm}$ :** Shannon entropy of the intensity distribution, inverted and normalized to  $[0, 1]$ ,

$$E_{norm} = 1 - \frac{1}{\log_2 L} \left( - \sum_{i=0}^{L-1} h(i) \log_2 h(i) \right). \quad (5)$$

Low entropy (little texture/information content) often co-occurs with under-exposure.

The frame is routed to the enhancement branch whenever the cumulative score exceeds a calibrated threshold  $\theta$ :

$$\text{Decision} = \begin{cases} \text{dark (enhance)} & Q(I_{raw}) \geq \theta \\ \text{normal (skip enhancement)} & Q(I_{raw}) < \theta \end{cases} \quad (6)$$

### 2.2.3 Conditional Branch – Low-Light Enhancement (Zero-DCE++)

If the frame is classified as *dark*, it is passed through **Zero-DCE++** (Zero-Reference Deep Curve Estimation), a lightweight, unsupervised image-enhancement network that learns a pixel-wise



higher-order curve mapping  $\mathcal{F}_\Theta$  without requiring paired low/normal-light training data. For an input pixel intensity  $x$ , the network iteratively applies a learned quadratic curve:

$$LE_n(x) = LE_{n-1}(x) + \alpha_n(x) LE_{n-1}(x) (1 - LE_{n-1}(x)), \quad n = 1, \dots, N_{iter} \quad (7)$$

where  $LE_0(x) = x$  is the original pixel value,  $\alpha_n(x) \in [-1, 1]$  is a pixel-wise curve parameter map predicted by a small CNN (*DCE-Net*), and  $N_{iter}$  controls the strength of enhancement. Zero-DCE++ is selected here because it:

- operates without ground-truth references, making it robust across arbitrary CCTV lighting conditions;
- is lightweight enough for near-real-time inference on edge/intersection hardware;
- empirically improves **lane-marking visibility** and **vehicle contour / pedestrian visibility** – both critical to the downstream detection task.

#### 2.2.4 Bilateral Denoising

Brightening dark regions (via curve estimation) tends to amplify sensor noise. The enhanced frame is therefore passed through a **bilateral filter**, which smooths noise while explicitly preserving edges – important for retaining lane lines, plate edges, and vehicle boundaries. For a pixel  $p$ :

$$I_{bf}(p) = \frac{1}{W_p} \sum_{q \in \Omega(p)} I_{LE}(q) G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_{LE}(p) - I_{LE}(q)|) \quad (8)$$

$$W_p = \sum_{q \in \Omega(p)} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_{LE}(p) - I_{LE}(q)|) \quad (9)$$

where  $\Omega(p)$  is a local neighborhood around  $p$ ,  $G_{\sigma_s}$  is a spatial Gaussian kernel (controls how far neighboring pixels contribute), and  $G_{\sigma_r}$  is a range/intensity Gaussian kernel (controls how strongly intensity-similar pixels contribute, suppressing blending across edges).

#### 2.2.5 Deblurring (Wiener Deconvolution)

Both branches – the enhanced-and-denoised path *and* the path for frames already classified as *normal* – converge at a single, mandatory deblurring step. Motion blur from vehicles crossing the intersection is modeled as a linear, spatially invariant degradation:

$$g(x, y) = (f * h)(x, y) + n(x, y) \quad (10)$$

where  $f$  is the latent sharp image,  $h$  is the (motion) point-spread function (PSF),  $n$  is additive noise, and  $*$  denotes 2-D convolution. In the frequency domain this becomes  $G(u, v) = F(u, v)H(u, v) + N(u, v)$ . The **Wiener deconvolution** filter recovers an estimate  $\hat{F}(u, v)$  that minimizes the mean-squared error between the estimate and the true image:

$$\hat{F}(u, v) = \left[ \frac{H^*(u, v)}{|H(u, v)|^2 + \frac{S_n(u, v)}{S_f(u, v)}} \right] G(u, v) \quad (11)$$

where  $H^*(u, v)$  is the complex conjugate of the PSF's Fourier transform, and  $S_n(u, v)/S_f(u, v)$  is the noise-to-signal power ratio (often approximated by a single regularization constant  $K$  when per-frequency spectra are unavailable, giving the commonly implemented form  $\hat{F}(u, v) = \frac{H^*}{|H|^2 + K} G(u, v)$ ). The deblurred image  $\hat{f} = \mathcal{F}^{-1}\{\hat{F}\}$  is what is finally handed to the detector.

### 2.3 Algorithmic Summary – Stage 1

```

1 import cv2
2 import numpy as np
3
4 # ---- Tunable weights (Eq. 1) and threshold ----
5 W1, W2, W3, W4 = 0.25, 0.25, 0.35, 0.15 # histogram, dark-pct,
    brightness, entropy
6 THRESHOLD = 0.55 # darkness gate, tuned
    empirically
7 DARK_PIXEL_TAU = 50 # 0-255 scale
8
9 def histogram_spread(gray):
10     hist, _ = np.histogram(gray, bins=256, range=(0, 256), density=True)
11     levels = np.arange(256)
12     mu = np.sum(levels * hist)
13     sigma = np.sqrt(np.sum(hist * (levels - mu) ** 2))
14     return sigma / 255.0
15
16 def pct_dark_pixels(gray, tau=DARK_PIXEL_TAU):
17     return float(np.mean(gray < tau))
18
19 def mean_brightness_score(gray):
20     return 1.0 - (gray.mean() / 255.0)
21
22 def normalized_entropy_score(gray):
23     hist, _ = np.histogram(gray, bins=256, range=(0, 256), density=True)
24     hist = hist[hist > 0]
25     entropy = -np.sum(hist * np.log2(hist))
26     return 1.0 - (entropy / np.log2(256))
27
28 def darkness_score(frame_bgr):
29     gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
30     h_spread = histogram_spread(gray)
31     p_dark = pct_dark_pixels(gray)
32     b_mean = mean_brightness_score(gray)
33     e_norm = normalized_entropy_score(gray)
34     Q = W1 * h_spread + W2 * p_dark + W3 * b_mean + W4 * e_norm
35     return Q
36
37 def zero_dce_pp(frame_bgr, model):
38     """Pixel-wise curve enhancement (DCE-Net inference); model loaded
    upstream."""
39     return model.infer(frame_bgr)
40
41 def bilateral_denoise(frame_bgr, d=9, sigma_color=75, sigma_space=75):
42     return cv2.bilateralFilter(frame_bgr, d, sigma_color, sigma_space)

```

```
43
44 def wiener_deconvolution(frame_bgr, psf, K=0.01):
45     """Per-channel Wiener deconvolution in the frequency domain (Eq. 9)."""
46     out = np.zeros_like(frame_bgr, dtype=np.float32)
47     psf_fft = np.fft.fft2(psf, s=frame_bgr.shape[:2])
48     psf_conj = np.conj(psf_fft)
49     denom = (np.abs(psf_fft) ** 2) + K
50     for c in range(3):
51         G = np.fft.fft2(frame_bgr[:, :, c])
52         F_hat = (psf_conj / denom) * G
53         out[:, :, c] = np.real(np.fft.ifft2(F_hat))
54     return np.clip(out, 0, 255).astype(np.uint8)
55
56 def preprocess_frame(frame_bgr, zero_dce_model, psf):
57     Q = darkness_score(frame_bgr)
58     if Q >= THRESHOLD:                                     # "if (score < threshold):
59         image is dark?" -> yes
60         enhanced = zero_dce_pp(frame_bgr, zero_dce_model)
61         denoised = bilateral_denoise(enhanced)
62         deblurred = wiener_deconvolution(denoised, psf)
63     else:                                                   # no -> skip enhancement
64         branch
65         deblurred = wiener_deconvolution(frame_bgr, psf)
66     return deblurred    # ready for YOLO anomaly detection
```

Listing 1: Pre-anomaly-detection preprocessing pipeline

### 3 Stage 2: Post-Anomaly-Detection Pipeline

#### 3.1 Pipeline Diagram

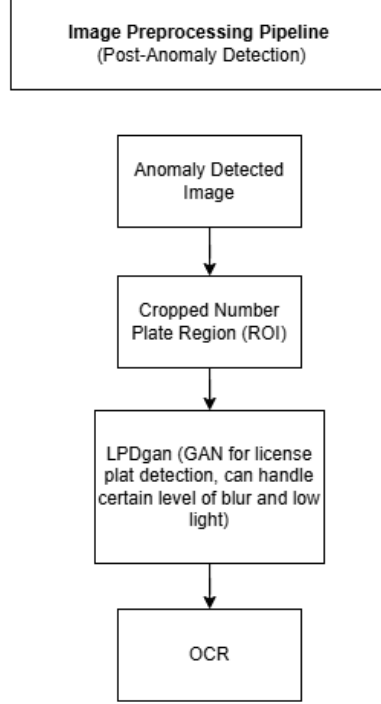


Figure 2: Post-Anomaly Image Pre-Processing

#### 3.2 Stage Description

##### 3.2.1 Region-of-Interest (ROI) Cropping

Once Stage 1 (fused with the violation-classification logic) flags a vehicle as an anomaly (any of 7 identified anomalies) its bounding box from the YOLO detector is used to localize and crop the **number-plate region**:

$$I_{ROI} = I_{enh}[y_0 : y_1, x_0 : x_1] \quad (12)$$

where  $(x_0, y_0, x_1, y_1)$  is the plate sub-region within (or derived from) the vehicle's bounding box, typically obtained from a dedicated plate-localization head or a fixed geometric prior relative to the vehicle box. Cropping restricts all subsequent, more expensive restoration computation to a small region rather than the full frame.

##### 3.2.2 Plate Restoration – LPDGAN

$I_{ROI}$  is frequently still degraded – by residual motion blur (vehicle was moving at the violation instant) and/or low light – even after Stage 1's frame-level correction, since frame-level deblurring parameters are tuned for the whole scene rather than the small, fast-moving plate region.

**LPDGAN** (License-Plate-Deblurring GAN) is a conditional generative-adversarial network specialized for this restoration sub-task. Its generator  $\mathcal{G}$  produces a restored plate crop, while a discriminator  $\mathcal{D}$  is trained adversarially to distinguish restored crops from real sharp plates, using the standard conditional-GAN minimax objective:

$$\min_{\mathcal{G}} \max_{\mathcal{D}} \mathbb{E}_{I_{sharp}} [\log \mathcal{D}(I_{sharp})] + \mathbb{E}_{I_{ROI}} [\log(1 - \mathcal{D}(\mathcal{G}(I_{ROI})))] \quad (13)$$

augmented at training time with a pixel-level reconstruction term (e.g.  $L_1$  loss against the ground-truth sharp plate) to keep the restored output faithful to the actual characters rather than merely "plausible-looking":

$$\mathcal{L}_{total} = \mathcal{L}_{GAN} + \lambda \cdot \|\mathcal{G}(I_{ROI}) - I_{sharp}\|_1 \quad (14)$$

LPDGAN is chosen at this stage specifically because it can **handle a certain level of residual blur and low light** that survives the upstream Stage 1 corrections – i.e., it acts as a second, targeted restoration pass rather than a duplicate of the frame-level enhancement.

### 3.3 Algorithmic Summary – Stage 2

```

1 import cv2
2
3 def crop_plate_roi(frame, plate_bbox):
4     """plate_bbox: (x0, y0, x1, y1) from the plate-localization head."""
5     x0, y0, x1, y1 = plate_bbox
6     return frame[y0:y1, x0:x1]
7
8 def lpdgan_restore(plate_crop, generator_model):
9     """
10    Single forward pass through the LPDGAN generator G.
11    Handles moderate residual blur / low-light left after Stage 1.
12    """
13    restored = generator_model.infer(plate_crop)
14    return restored
15
16 def run_ocr(restored_plate, ocr_engine):
17     return ocr_engine.recognize(restored_plate)
18
19 def postprocess_anomaly(frame, plate_bbox, generator_model, ocr_engine):
20     roi = crop_plate_roi(frame, plate_bbox)
21     restored = lpdgan_restore(roi, generator_model)
22     plate_str = run_ocr(restored, ocr_engine)
23     return plate_str

```

Listing 2: Post-anomaly-detection plate restoration and OCR pipeline

## 4 End-to-End Pipeline

The two stages are intentionally decoupled: Stage 1 runs on *every* frame to support reliable detection, whereas Stage 2 is invoked *only* for the comparatively rare subset of vehicles classified as violations – keeping the expensive GAN-based restoration off the real-time critical path.

## 5 Important Logic Points

- **Quality-gated enhancement.** Zero-DCE++ is invoked conditionally, not unconditionally – only frames whose cumulative darkness score  $Q$  meets or exceeds  $\theta$  enter the enhancement branch (Eq. 6). This avoids over-processing already well-lit frames.
- **Weighted, multi-statistic darkness score.** No single statistic (brightness alone, say) is trusted;  $Q$  combines histogram spread, dark-pixel fraction, mean brightness, and entropy, weighted by  $w_1, \dots, w_4$  (Eq. 1), making the gate robust to corner cases (e.g. a bright headlight glare against an otherwise dark scene).
- **Deblurring is unconditional.** Unlike the brightness branch, Wiener deconvolution (Eq. 11) is applied to *every* frame regardless of the darkness decision, since motion blur is caused by vehicle/camera dynamics, not illumination.
- **Denoising follows enhancement, not before it.** Bilateral filtering is applied *after* Zero-DCE++ specifically because curve-based brightening amplifies sensor noise that was previously masked in dark regions.
- **Stage 2 is conditionally triggered.** The plate-restoration sub-pipeline (ROI crop  $\rightarrow$  LPDGAN  $\rightarrow$  OCR) only executes for frames/vehicles that the anomaly logic has already flagged as violations, keeping per-frame computation low for the (majority) non-violating traffic.
- **LPDGAN complements, not duplicates, Stage 1.** Stage 1 corrects the *whole frame*; LPDGAN performs a second, targeted restoration pass on the small plate crop, tolerating residual blur/low light that frame-level correction could not fully resolve.
- **Modularity.** Each block (Zero-DCE++, bilateral filter, Wiener deconvolution, YOLO, LPDGAN, OCR) is independently swappable, allowing the pipeline to be re-tuned or re-trained block-by-block as better models become available.

## 6 Summary

The Image Preprocessing Pipeline applies a two-stage, condition-aware enhancement strategy: a quality-gated, frame-level correction (darkness scoring  $\rightarrow$  optional Zero-DCE++  $\rightarrow$  bilateral denoise  $\rightarrow$  mandatory Wiener deblur) before anomaly detection, and a targeted, GAN-based plate restoration (LPDGAN) after a violation is flagged, immediately prior to OCR. This division of labor keeps real-time detection efficient while still providing high-fidelity restoration exactly where it is needed most – the license-plate region of a confirmed violator.

## References

- [1] R. C. Gonzalez and R. E. Woods, [https://www.c172.org/090imagePLib/books/Gonzales, Woods-Digital.Image.Processing.4th.Edition.pdf](https://www.c172.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf), 4th edition, Pearson, 2018.
- [2] Github reference for Zero-DCE <https://github.com/Li-Chongyi/Zero-DCE>
- [3] C. Guo et al., “Zero-DCE++: Efficient Deep Curve Estimation for Low-Light Image Enhancement,” <https://arxiv.org/pdf/2001.06826>, vol. 44, no. 10, pp. 7285–7298, 2022.
- [4] Github refernce for LPDgan – <https://github.com/haoyGONG/LPDGAN>

# A Detection, Tracking, and Crop-Extraction Pipeline for Multi-Class Traffic Violation Monitoring

## 1 Detection and Tracking

### 1.1 Detector

Object localization is performed by YOLO26, the Ultralytics detection model family. YOLO26 is pretrained on the COCO dataset (80 object classes) and produces bounding boxes directly without a separate non-maximum-suppression post-processing stage, which reduces per-frame latency relative to earlier YOLO generations. The pipeline uses the nano-scale checkpoint (`yolo26n.pt`), the variant intended for CPU- and edge-constrained deployment, with the understanding that a larger checkpoint (`yolo26s`, `yolo26m`) trades latency for improved recall on small or distant objects.

The detector is queried once per frame and returns, for every detected object, a bounding box in pixel coordinates, a class label drawn from the COCO class set, and a confidence score.

### 1.2 Tracker

Detections are linked across frames using BoT-SORT, the default multi-object tracker shipped with Ultralytics. BoT-SORT extends ByteTrack, a motion-and-IoU-based tracker, with two additional mechanisms that are relevant to a scene with a non-static or vibrating camera and with visually similar objects in close proximity:

- **Global Motion Compensation (GMC).** Before each frame’s detections are matched to existing tracks, an affine transform describing the camera’s own motion between the previous and current frame is estimated and applied to every track’s predicted (Kalman-filter) state, preventing camera shake or pan from being misread as object motion. The transform is estimated from sparse keypoints tracked between frames, with outlier correspondences rejected before an affine fit.
- **Re-Identification (ReID).** Optionally, an appearance embedding is extracted for every detected box and fused into the cost matrix used for frame-to-frame matching, alongside the existing motion/IoU cost. This reduces identity switches when two tracked objects of the same class pass close to one another.

Tracking proceeds as a per-frame assignment problem. For each frame  $t$ , let  $\mathcal{T}_{t-1} = \{T_1, \dots, T_m\}$  be the active tracks carried forward from the previous frame, and let  $\mathcal{D}_t = \{d_1, \dots, d_n\}$  be the detections on frame  $t$ . Each track  $T_i$  carries a predicted state  $\hat{x}_i$  from a Kalman filter, after the GMC warp has been applied. A cost  $c_{ij}$  is computed between every track  $T_i$  and detection  $d_j$  as a combination of spatial distance (one minus IoU between predicted and detected boxes) and, if ReID is enabled, appearance distance (one minus cosine similarity between embeddings). Assignment is solved as a linear bipartite matching problem (the Hungarian algorithm) over  $c_{ij}$ , subject to a maximum-cost cutoff below which a match is accepted. Unmatched detections seed new tracks; unmatched tracks are retained as *lost* for a configurable number of frames, in case the object reappears after brief occlusion, before being terminated.

### 1.3 Tracker Configuration

The tracker is configured programmatically rather than from a static file, ensuring the configuration is generated identically on every run regardless of operating system or text-encoding quirks. Table 1 lists the parameters used and their role in the matching process described above.

### 1.4 Implementation

The configuration is written to a YAML file at runtime, the video stream is opened with its native frame rate to preserve correct playback timing in the annotated output, and every frame is passed through the tracker with `persist=True`, which instructs the tracker to retain track state across the individual, separately-supplied frames rather than resetting at every call.

```
import yaml
from ultralytics import YOLO
from ultralytics.data.loaders import get_best_youtube_url
```

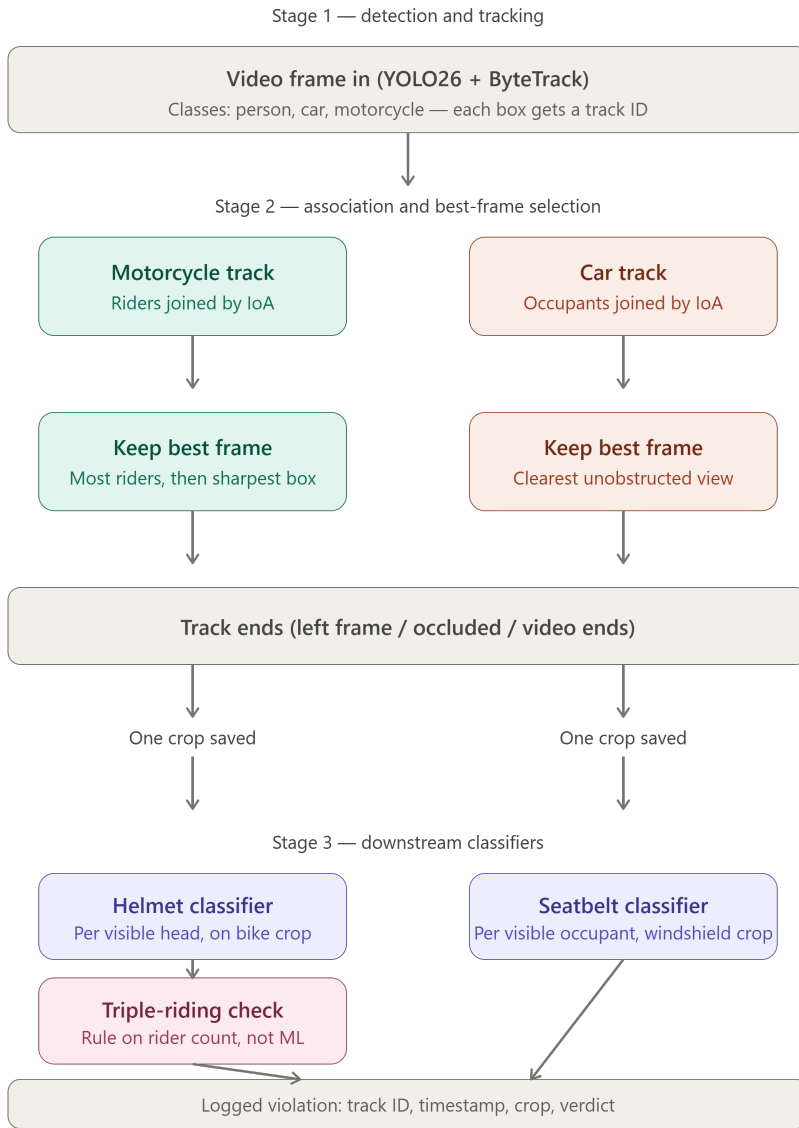


Figure 1: Workflow of multi-object detection and tracking pipeline.



| Parameter                              | Role                                                                    |
|----------------------------------------|-------------------------------------------------------------------------|
| <code>tracker_type: botsort</code>     | Selects BoT-SORT over the simpler ByteTrack baseline.                   |
| <code>track_high_thresh: 0.25</code>   | Confidence floor for a detection to enter the first association pass.   |
| <code>track_low_thresh: 0.1</code>     | Confidence floor for the second-pass association of weaker detections.  |
| <code>new_track_thresh: 0.25</code>    | Minimum confidence for an unmatched detection to start a new track.     |
| <code>track_buffer: 60</code>          | Frames a track is kept as <i>lost</i> before termination.               |
| <code>match_thresh: 0.8</code>         | Maximum acceptable cost $c_{ij}$ for a valid track–detection match.     |
| <code>fuse_score: True</code>          | Blends detection confidence into the IoU cost before matching.          |
| <code>gmc_method: sparseOptFlow</code> | Camera-motion estimation via sparse optical flow keypoints.             |
| <code>proximity_thresh: 0.5</code>     | Minimum IoU for a track–detection pair to be eligible for ReID scoring. |
| <code>appearance_thresh: 0.8</code>    | Minimum embedding similarity required for a ReID-based match.           |
| <code>with_reid: True</code>           | Enables appearance-embedding fusion into the cost matrix.               |

Table 1: BoT-SORT configuration parameters and their function in the assignment process.

```
# Generated programmatically to avoid encoding/formatting
# inconsistencies across platforms.
tracker_config = {
    "tracker_type": "botsort",
    "track_high_thresh": 0.25,
    "track_low_thresh": 0.1,
    "new_track_thresh": 0.25,
    "track_buffer": 60,
    "match_thresh": 0.8,
    "fuse_score": True,
    "gmc_method": "sparseOptFlow",
    "proximity_thresh": 0.5,
    "appearance_thresh": 0.8,
    "with_reid": True,
}
custom_tracker_path = "custom_botsort.yaml"
with open(custom_tracker_path, "w") as f:
    yaml.dump(tracker_config, f)

model = YOLO("yolo26n.pt")
video_url = get_best_youtube_url(
    "https://www.youtube.com/shorts/4im-GdrR2w8", method="yt-dlp"
)

# cap = cv2.VideoCapture(video_url); frame-read loop omitted (I/O only)
# persist=True retains track identities across the individually
# supplied frames; class filtering is NOT applied here -- every COCO
# class is tracked, and is filtered downstream at the cropping stage.
results = model.track(
    frame, persist=True, tracker=custom_tracker_path, verbose=False
)
```

## 2 Crop Extraction

### 2.1 Problem Statement

A raw track produces one bounding box per frame for as long as the object remains visible – typically several dozen to several hundred frames for a vehicle crossing a camera’s field of view. Passing every one of those frames to a downstream classifier would be both computationally wasteful and a source of inconsistent verdicts for what is a single physical event (a single motorcycle’s compliance status does not change between consecutive frames; only the visibility and clarity of the riders does). The crop-extraction stage therefore reduces each vehicle track to exactly one image: the single frame, over that vehicle’s entire visible lifetime, in which the vehicle and its occupants are most fully and clearly visible.

This requires two pieces of logic: a rule for deciding which detected people belong to which vehicle, and a rule for scoring frames against one another so the best one can be retained and emitted when the track ends.

### 2.2 Occupant Association

For a given vehicle box  $B_v = (x_1, y_1, x_2, y_2)$  and a candidate person box  $B_p$ , the association rule is based on Intersection over the person box’s own Area (IoA), rather than Intersection over Union:

$$\text{IoA}(B_p, B_v) = \frac{\text{Area}(B_p \cap B_v)}{\text{Area}(B_p)} \quad (1)$$

A person is associated with a vehicle when  $\text{IoA}(B_p, B_v) > \tau$  for a threshold  $\tau$ . IoA is used in place of IoU because the question being asked is "what fraction of this person's body lies inside the vehicle," not "how similarly sized and positioned are these two boxes." A standing adult and a small motorcycle can have a low IoU despite the person clearly riding it, since the person's box is much larger than the bike's; IoA is invariant to this size mismatch.

For a motorcycle,  $B_v$  is the full detected bounding box, since a seated rider's body legitimately overlaps the entire box. A motorcycle track may have zero, one, two, or more associated riders in a given frame; this is what allows the same association logic to support a triple-riding check downstream without any additional model.

For a car, occupants are only partially visible – the lower body is occluded by the vehicle shell, and the relevant compliance signal (a seatbelt strap) is confined to the upper body in any case. Association for cars is therefore computed against a restricted zone  $Z_v$  taken from the upper fraction  $f$  of the car's box:

$$Z_v = (x_1, y_1, x_2, y_1 + f \cdot (y_2 - y_1)) \quad (2)$$

with  $\text{IoA}(B_p, Z_v)$  used in place of  $\text{IoA}(B_p, B_v)$  for the car case. The implementation below shows the motorcycle path; the car path follows the identical structure with the vehicle box replaced by  $Z_v$  and is omitted from the listing, since car crops are routed to a separately named classifier outside this document's scope.

### 2.3 Best-Frame Selection

Let track  $k$  accumulate, over its lifetime, a sequence of observations  $(f_1^{(k)}, f_2^{(k)}, \dots)$ , where each observation at frame  $t$  consists of the vehicle box, the set of associated rider boxes  $R_t = \{B_p^1, \dots, B_p^{n_t}\}$ , and the frame image itself. Two quantities are computed at each observation: the rider count  $n_t = |R_t|$ , and a clarity proxy  $s_t$  defined as the area of the smallest box enclosing the vehicle and all its associated riders:

$$s_t = \text{Area}\left(\bigcup (\{B_v\} \cup R_t)\right) \quad (3)$$

where  $\bigcup(\cdot)$  denotes the smallest axis-aligned box containing every box in the set. The observation at frame  $t$  replaces the best observation retained so far if and only if

$$n_t > n_{\text{best}} \quad \vee \quad (n_t = n_{\text{best}} \wedge s_t > s_{\text{best}}) \quad (4)$$

Rider count is the primary key and is never traded off against clarity: a frame showing all three riders on an overloaded motorcycle must be preferred over a sharper frame that happens to catch only two of them, since under-counting riders would cause a true triple-riding violation to be missed by the downstream check. Clarity, measured as the union box's pixel area, is used only to break ties among frames with equal rider count, as a proxy for a closer, less occluded, less motion-blurred view.

### 2.4 Track Lifetime and Emission

A track is considered finished – and its best observation is emitted as a single crop – when the tracker reports no detection for that track ID for more than a fixed number of consecutive frames,  $\Delta_{\text{timeout}}$ . This covers a vehicle leaving the frame, a brief full occlusion that exceeds the tracker's own re-association window, or the end of the video. Tracks still open at the end of the video are flushed using whatever best observation they accumulated. This guarantees exactly one emitted crop per physical vehicle appearance, regardless of how many frames that vehicle was visible for.

The emitted crop is the union box of the vehicle and its associated riders at the best observation, padded by a fixed margin in pixels and clipped to the frame boundary.

### 2.5 Implementation

```
import numpy as np

PERSON_CLASS = 0
MOTORCYCLE_CLASS = 3
```

```

# CAR_CLASS = 2 is handled by the same logic against a restricted
# upper-body zone of the car box, and is omitted here; car crops are
# routed to a separately named seatbelt-compliance classifier.

IOA_THRESHOLD = 0.30
CROP_MARGIN = 10
TRACK_TIMEOUT_FRAMES = 15

def calculate_ioa(person_box, vehicle_box):
    px1, py1, px2, py2 = person_box
    vx1, vy1, vx2, vy2 = vehicle_box

    x_left, y_top = max(px1, vx1), max(py1, vy1)
    x_right, y_bottom = min(px2, vx2), min(py2, vy2)
    if x_right <= x_left or y_bottom <= y_top:
        return 0.0

    intersection = (x_right - x_left) * (y_bottom - y_top)
    person_area = (px2 - px1) * (py2 - py1)
    return intersection / person_area if person_area > 0 else 0.0

def union_box(boxes):
    boxes = np.array(boxes)
    return (boxes[:, 0].min(), boxes[:, 1].min(),
            boxes[:, 2].max(), boxes[:, 3].max())

def box_area(box):
    x1, y1, x2, y2 = box
    return max(0.0, x2 - x1) * max(0.0, y2 - y1)

class TrackState:
    """Best observation seen so far for one motorcycle track."""

    def __init__(self):
        self.rider_count = -1
        self.score = -1.0
        self.frame = None
        self.bike_box = None
        self.rider_boxes = None
        self.last_seen_frame = 0

    def maybe_update(self, frame_idx, frame, bike_box, rider_boxes):
        rider_count = len(rider_boxes)
        combined = [bike_box] + rider_boxes if rider_boxes else [bike_box]
        score = box_area(union_box(combined))

        is_better = (
            self.frame is None
            or rider_count > self.rider_count
            or (rider_count == self.rider_count and score > self.score)
        )
        if is_better:
            self.rider_count = rider_count
            self.score = score
            self.frame = frame.copy()
            self.bike_box = bike_box
            self.rider_boxes = rider_boxes
            self.last_seen_frame = frame_idx

def save_crop(track_id, state, output_dir, frame_shape):
    if state.frame is None:
        return
    h, w = frame_shape[:2]
    x1, y1, x2, y2 = union_box([state.bike_box] + state.rider_boxes)

    x1 = max(0, int(x1) - CROP_MARGIN)
    y1 = max(0, int(y1) - CROP_MARGIN)
    x2 = min(w, int(x2) + CROP_MARGIN)
    y2 = min(h, int(y2) + CROP_MARGIN)

    crop = state.frame[y1:y2, x1:x2]
    # crop is written to disk and handed to the helmet / triple-riding
    # classifiers; that step is outside the scope of this document.

```

The per-frame driver loop associates people to motorcycles via `calculate_ioa`, updates each track's `TrackState` via `maybe_update`, and flushes (`save_crop`) any track absent from the current frame's detections for more than

TRACK.TIMEOUT.FRAMES consecutive frames, mirroring the lifetime rule of Section 3.4. The identical structure, applied to car tracks against the restricted upper-body zone  $Z_v$ , produces car crops for the seatbelt-compliance classifier.

### 3 Downstream Violation Classification

Following the extraction of optimal per-vehicle crops, the pipeline routes these images to specific downstream modules designed to identify localized traffic violations. These modules operate independently on the localized regions of interest, benefiting from the reduced background noise and optimal clarity provided by the extraction stage.

#### 3.1 Helmet Detection

For motorcycle tracks, the emitted crops containing the vehicle and its associated riders are passed to a secondary object detection model fine-tuned for helmet compliance. This model evaluates whether the bounding boxes of the detected occupants contain safety helmets. The underlying model is developed and trained utilizing the publicly available Helmet Detection dataset (<https://www.kaggle.com/datasets/andrewmvd/helmet-detection/data>), which provides extensive bounding box annotations for both helmeted and non-helmeted riders across varying conditions.

#### 3.2 Seatbelt Detection

Similarly, crops extracted from the restricted upper-body zone  $Z_v$  of tracked cars are routed to a dedicated seatbelt-compliance classifier. This module inspects the visible passenger cabin for the presence of an engaged seatbelt strap across the vehicle’s occupants. The model for this task is built upon the Seatbelt Dataset (<https://www.kaggle.com/datasets/sachinxshrivastav/seatbelt-dataset>), enabling the system to reliably discriminate between compliant and non-compliant drivers within the isolated vehicle window.

#### 3.3 Triple-Riding Detection

Unlike helmet and seatbelt compliance, which require secondary visual feature extraction, triple-riding detection is resolved entirely by the geometric association logic established during the crop-extraction phase. As detailed in Section 2, the pipeline already evaluates the spatial relationship between person boxes and motorcycle boxes using the Intersection over Area (IoA) criterion.

Because the association logic accurately links overlapping riders to a specific motorcycle bounding box, the system simply retrieves the associated occupant count  $n_{\text{best}}$  from the emitted optimal frame. A triple-riding violation is deterministically flagged if the count of associated riders satisfies  $n_{\text{best}} \geq 3$ . This approach eliminates the computational overhead of running an additional dense-prediction model, directly leveraging the robust spatial relationships and tracking history already derived from the primary detection stage.

## 4 Summary

The pipeline described in this document converts raw video into one labeled crop per vehicle appearance: a bounding box, an associated set of occupants, and the single clearest frame across that vehicle’s tracked lifetime. The detection stage (YOLO26) and tracking stage (BoT-SORT, with global motion compensation and optional re-identification) together provide persistent per-object identity under camera motion and visual ambiguity. The association and best-frame-selection logic, expressed through the IoA criterion and the rider-count-then-clarity ordering, converts that raw tracking output into exactly the unit of input – one crop, fully representing one vehicle and its occupants – required by the downstream helmet, triple-riding, and seatbelt classifiers.

## Sources

- Ultralytics multi-object tracking documentation: <https://docs.ultralytics.com/modes/track>
- Ultralytics GitHub repository: <https://github.com/ultralytics/ultralytics>
- BoT-SORT tracker configuration reference: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/trackers/botsort.yaml>
- Ultralytics YOLO26 model documentation: <https://docs.ultralytics.com/models/yolo26>

# Vehicle Speed Estimation via Multi-Object Tracking and Planar Homography

## 1 Pipeline Overview

The pipeline consists of five sequential stages applied to every frame of a video captured at known frame rate  $f_{\text{fps}}$ :

1. **Tracking:** assigning a unique, persistent identity  $k$  and bounding box to each vehicle via BoT-SORT.
2. **Reference extraction:** extracting the ground-contact point of each tracked vehicle.
3. **Perspective projection:** mapping the 2D pixel coordinate to a 2D real-world metric coordinate via homography.
4. **Plausibility gating:** rejecting displacements that imply a physically impossible speed, treating them as tracker discontinuities rather than real motion.
5. **Speed computation & smoothing:** calculating displacement over a sliding window and applying an Exponential Moving Average (EMA) to suppress detection jitter.

## 2 Algorithmic Formulation

### 2.1 Reference Point Localization

For a bounding box defined by  $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ , the chosen reference point is the **ground-contact centroid** (bottom-center of the box):

$$p_k^{(t)} = \left( \frac{x_{\min} + x_{\max}}{2}, y_{\max} \right) \quad (1)$$

*Justification:* the homography of Section 2.2 is valid only for points lying on the calibration plane (the road surface). The bottom-center of the box is the closest 2D approximation to the vehicle’s true wheel-to-road contact location available from an axis-aligned bounding box.

### 2.2 Perspective Calibration (Homography)

A homography matrix  $H \in \mathbb{R}^{3 \times 3}$  maps a homogeneous pixel coordinate  $(u, v, 1)^T$  to a world coordinate in meters:

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad (X, Y) = \left( \frac{x'}{w}, \frac{y'}{w} \right) \quad (2)$$

**Why four points suffice.** Fixing  $h_{33} = 1$  leaves 8 unknown entries in  $H$ . Each known correspondence  $(u_i, v_i) \mapsto (X_i, Y_i)$  contributes two linear equations:

$$X_i(h_{31}u_i + h_{32}v_i + 1) = h_{11}u_i + h_{12}v_i + h_{13} \quad (3)$$

$$Y_i(h_{31}u_i + h_{32}v_i + 1) = h_{21}u_i + h_{22}v_i + h_{23} \quad (4)$$

Four correspondences yield exactly 8 equations in 8 unknowns, solved directly by the OpenCV function `getPerspectiveTransform`. The four reference points and their real-world correspondences (a known lane width and segment length) are fixed once per camera setup, as shown in Listing 1.

### 2.3 Plausibility Gating

A tracker identity switch or a re-identification failure following occlusion produces a discontinuous jump in  $P_k^{(t)}$  that is not real vehicle motion. Such a jump is detected *before* it enters the trajectory history, using the displacement to the immediately preceding point:

$$\text{reject if } \frac{d_k(t-1, t)}{1/f_{\text{fps}}} \times 3.6 > v_{\text{max}} \quad (5)$$

On rejection, the implementation clears the identity’s history rather than discarding only the offending point, since a corrupted track should not silently resume averaging once the discontinuity has occurred.

### 2.4 Speed Calculation and Temporal Smoothing

Once at least  $w = 15$  points of (gated) history exist for identity  $k$ , the Euclidean ground-plane displacement across the window is:

$$d_k(t-w, t) = \sqrt{\left(X_k^{(t)} - X_k^{(t-w)}\right)^2 + \left(Y_k^{(t)} - Y_k^{(t-w)}\right)^2} \quad (6)$$

with elapsed time taken from the video’s native frame rate rather than wall-clock processing time:

$$\Delta t = \frac{t - (t-w)}{f_{\text{fps}}} \quad (7)$$

giving the raw windowed speed in km/h:

$$v_k^{(t)} = \frac{d_k(t-w, t)}{\Delta t} \times 3.6 \quad (8)$$

which is smoothed across time with an Exponential Moving Average of factor  $\alpha \in (0, 1]$ :

$$\bar{v}_k^{(t)} = \alpha v_k^{(t)} + (1 - \alpha) \bar{v}_k^{(t-1)}, \quad \bar{v}_k^{(t_0)} = v_k^{(t_0)} \text{ at first valid estimate} \quad (9)$$

**Why smoothing is necessary even with a sliding window.** The window in Equation (6) re-evaluates its oldest endpoint every frame, so the raw estimate  $v_k^{(t)}$  still changes from frame to frame even when the vehicle moves at constant speed, due to bounding-box jitter at both window endpoints. The EMA of Equation (9) is therefore applied on top of windowing, not in place of it.

## 3 Code Implementation

The implementation below follows the Ultralytics YOLO + BoT-SORT structure used for detection and tracking, with the homography calibration, plausibility gate, and EMA smoothing integrated directly into the per-frame loop.

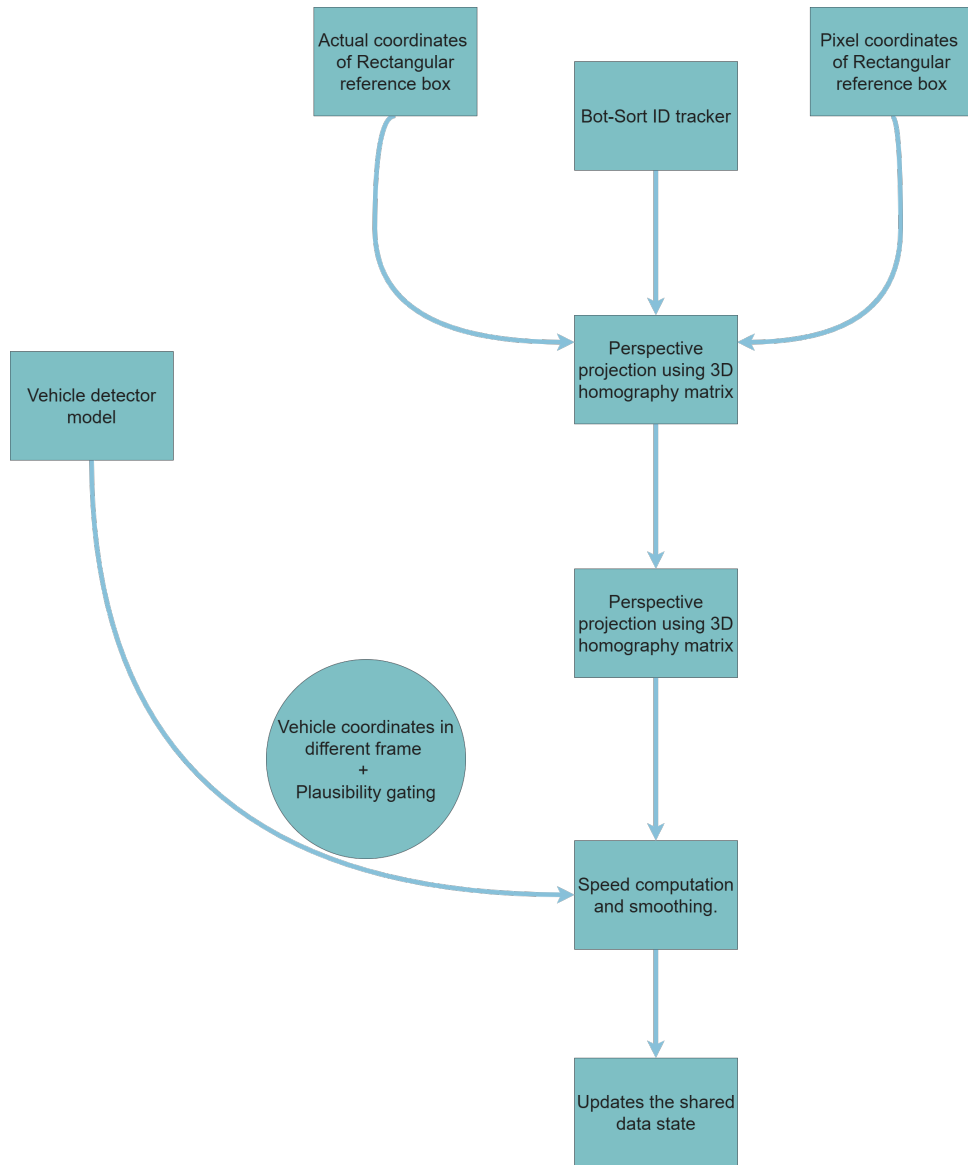


Figure 1: Workflow of the vehicle speed estimation pipeline.

### 3.1 Tracker Configuration and Homography Calibration

```

1 import cv2
2 import yaml
3 import numpy as np
4 from collections import defaultdict, deque
5 from ultralytics import YOLO
6 from ultralytics.data.loaders import get_best_youtube_url
7
8 # 1. Custom Tracker YAML
9 custom_tracker_path = "custom_botsort.yaml"
10 tracker_config = {
11     "tracker_type": "botsort",
12     "track_high_thresh": 0.25,
13     "track_low_thresh": 0.1,
14     "new_track_thresh": 0.25,
15     "track_buffer": 60,
16     "match_thresh": 0.8,

```

```

17     "fuse_score": True,
18     "gmc_method": "sparseOptFlow",
19     "proximity_thresh": 0.5,
20     "appearance_thresh": 0.8,
21     "with_reid": True
22 }
23
24 with open(custom_tracker_path, "w") as f:
25     yaml.dump(tracker_config, f)
26
27 # 2. Load the Model
28 model = YOLO("yolov8n.pt") #can change the model
29
30 # 3. Get Video Stream URL
31 video_url = get_best_youtube_url(
32     "https:your_url",
33     method="yt-dlp"
34 )
35
36 # 4. Video Setup
37 cap = cv2.VideoCapture(video_url)
38 if not cap.isOpened():
39     print("Error: Could not open video stream.")
40     exit()
41
42 w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
43 h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
44 fps = int(cap.get(cv2.CAP_PROP_FPS))
45 if fps == 0:
46     fps = 30
47
48 out_path = "tracked_speed_output.mp4"
49 fourcc = cv2.VideoWriter_fourcc(*"mp4v")
50 writer = cv2.VideoWriter(out_path, fourcc, fps, (w, h))
51
52 # --- Homography calibration: pixel polygon -> real-world rectangle ---
53 # IMPORTANT: replace these with coordinates matching a real rectangle
54 # on the road in your specific video.
55 src_points = np.array([
56     [100, 600], # Top-left pixel
57     [400, 600], # Top-right pixel
58     [500, 900], # Bottom-right pixel
59     [50, 900] # Bottom-left pixel
60 ], dtype=np.float32)
61
62 real_world_width = 3.5 # meters
63 real_world_length = 10.0 # meters
64
65 dst_points = np.array([
66     [0, 0],
67     [real_world_width, 0],
68     [real_world_width, real_world_length],
69     [0, real_world_length]
70 ], dtype=np.float32)
71
72 transform_matrix = cv2.getPerspectiveTransform(src_points, dst_points)
73
74 MAX_PLAUSIBLE_KMH = 200.0 # Eq. (4) gating threshold

```



```

75 EMA_ALPHA = 0.3 # Eq. (6) smoothing factor
76
77 track_history = defaultdict(lambda: deque(maxlen=fps))
78 smoothed_speed = {}

```

Listing 1: Tracker configuration and homography setup

### 3.2 Per-Frame Tracking and Speed Computation

```

1 frame_count = 0
2
3 try:
4     while cap.isOpened():
5         success, frame = cap.read()
6         if not success:
7             break
8
9         frame_count += 1
10
11         # tracker=custom_tracker_path wires in the BoT-SORT config built above.
12         # Without this argument, model.track() silently falls back to the
13         # default tracker and the tuned thresholds are never applied.
14         results = model.track(
15             frame, persist=True, verbose=False, tracker=custom_tracker_path
16         )
17         annotated_frame = frame.copy()
18
19         if results[0].boxes is not None and results[0].boxes.id is not None:
20             boxes = results[0].boxes.xyxy.cpu().numpy()
21             track_ids = results[0].boxes.id.cpu().numpy().astype(int)
22
23             for box, track_id in zip(boxes, track_ids):
24                 x1, y1, x2, y2 = box
25
26                 # Eq. (1): ground-contact reference point
27                 cx = (x1 + x2) / 2
28                 cy = y2
29
30                 # Eq. (2): pixel -> real-world via homography
31                 point_2d = np.array([[cx, cy]], dtype=np.float32)
32                 point_3d = cv2.perspectiveTransform(point_2d, transform_matrix)
33                 real_x, real_y = point_3d[0][0]
34
35                 # Eq. (4): plausibility gate, applied before appending history
36                 history = track_history[track_id]
37                 if len(history) > 0:
38                     last_x, last_y, last_frame = history[-1]
39                     step_distance = np.sqrt((real_x - last_x) ** 2 + (real_y - last_y)
40                                             ** 2)
41                     step_frames = frame_count - last_frame
42                     if step_frames > 0:
43                         implied_kmh = (step_distance / (step_frames / fps)) * 3.6
44                         if implied_kmh > MAX_PLAUSIBLE_KMH:
45                             history.clear()
46                             smoothed_speed.pop(track_id, None)
47
48                 history.append((real_x, real_y, frame_count))

```

```

48     speed_kmh = smoothed_speed.get(track_id, 0.0)
49     if len(history) >= 15:
50         old_x, old_y, old_frame = history[0]
51
52         # Eq. (5): windowed Euclidean displacement
53         distance_meters = np.sqrt((real_x - old_x) ** 2 + (real_y - old_y)
54                                   ** 2)
55
56         # Eq. (6): elapsed time from native fps
57         time_seconds = (frame_count - old_frame) / fps
58
59         if time_seconds > 0:
60             speed_ms = distance_meters / time_seconds
61             raw_speed_kmh = speed_ms * 3.6
62
63             # Eq. (7): EMA smoothing
64             prev = smoothed_speed.get(track_id)
65             speed_kmh = (
66                 raw_speed_kmh if prev is None
67                 else EMA_ALPHA * raw_speed_kmh + (1 - EMA_ALPHA) * prev
68             )
69             smoothed_speed[track_id] = speed_kmh
70
71     cv2.rectangle(annotated_frame, (int(x1), int(y1)), (int(x2), int(y2)),
72                  (0, 255, 0), 2)
73     cv2.putText(annotated_frame, f"ID: {track_id} | {speed_kmh:.1f} km/h",
74                (int(x1), int(y1) - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
75                255, 255), 2)
76
77     writer.write(annotated_frame)
78     cv2.imshow("Speed Tracking", annotated_frame)
79
80     if cv2.waitKey(1) & 0xFF == ord("q"):
81         break
82
83 finally:
84     cap.release()
85     if writer is not None:
86         writer.release()
87     cv2.destroyAllWindows()
88     print(f"Saved speed tracking video to {out_path}")

```

Listing 2: Main tracking loop with plausibility gating and EMA smoothing

## 4 Sources of Error and Limitations

- **Planarity assumption:** vehicles with high ground clearance, or camera angles that map onto a higher point on the vehicle body, introduce reprojection error, since the reference point of Equation (1) diverges from the true road-plane contact point as vehicle height increases relative to camera elevation.
- **Frame rate variability:** variable frame rate (VFR) source video invalidates the constant- $f_{\text{fps}}$  assumption in Equation (7). With VFR sources, per-frame timestamps from the video container should replace frame-index-based timing.

- **Residual ID-switch risk:** the plausibility gate of Section 2.3 catches large, obviously implausible jumps, but a moderate ID switch between two adjacent, similarly-positioned vehicles can fall below  $v_{\max}$  and still corrupt a window silently. This is a known residual risk rather than a fully solved case.
- **Calibration sensitivity:** error in the manually measured real-world reference distances used in Section 2.2 propagates directly into every downstream speed estimate; recalibration is required if the camera is repositioned.

## 5 References

- [https://github.com/roboflow/supervision/tree/develop/examples/speed\\_estimation](https://github.com/roboflow/supervision/tree/develop/examples/speed_estimation)
- <https://github.com/roboflow/notebooks/blob/main/notebooks/how-to-estimate-vehicle-speed.ipynb>
- <https://blog.roboflow.com/estimate-speed-computer-vision/>

# Illegal Parking and Wrong-Side Driving Detection System

## Technical Documentation & Logic Specification

Zone-based, motion-fused detection of two distinct roadside anomalies from a single per-vehicle decision pipeline: stationary vehicles dwelling in a marked no-parking zone beyond a class-dependent time threshold, and vehicles moving against the locally defined direction of travel.

## 1 Overview

The **Illegal Parking and Wrong-Side Driving (IPWSD)** module evaluates each tracked vehicle, frame by frame, using a branched decision pipeline driven by the vehicle’s instantaneous speed, its position relative to pre-segmented zones, and the sign of its motion along the locally defined direction axis. The overall flowchart is illustrated in Figure 1.

The module receives a pre-processed, enhanced CCTV frame from the image enhancement stage. From this single input, two independent anomaly checks are branched: a *stationary-vehicle* branch that tests for illegal parking, and a *moving-vehicle* branch that tests for wrong-side driving. The split is governed by a single decision: whether the vehicle’s speed is zero.

If the vehicle is stationary (speed = 0), the module checks whether the vehicle’s centroid lies inside a pre-marked no-parking zone, and if so whether it has dwelled there for longer than a class-dependent offset time  $t_{\text{offset}}$  (nominally 10 s for cars, 60 s for buses, to allow for brief, legitimate halts such as passenger drop-off). Exceeding this threshold inside the zone results in an **Illegal Parking** classification; otherwise the vehicle is marked **No Anomaly**.

If the vehicle is moving (speed  $\neq 0$ ), the module instead checks the sign relationship between the vehicle’s lateral position  $x$  and its signed speed against a pre-defined convention for that road segment (e.g.,  $x < 0 \Rightarrow$  speed should be positive, and vice versa). A mismatch against this defined sign convention indicates the vehicle is travelling against the permitted direction and is classified as **Wrong-Side Driving**; a consistent sign is classified as **No Anomaly**.

Both branches reuse the same enhanced frame and the same upstream-segmented zone map, so no additional image processing or model inference is required beyond what has already been computed for tracking and zone segmentation.

## 2 Stage-by-Stage Description

The pipeline takes the pre-processed, enhanced frame  $I_{\text{enh}}$  as input, together with per-vehicle tracking outputs (centroid, speed) assumed already computed upstream.

### 2.1 Zone and Sign-Convention Segmentation

No-parking zones are marked in the image ahead of time. The same segmentation pass also marks zones for which a direction convention applies, recording for each such zone whether motion along  $+x$  or  $-x$  is the legal direction. Formally, the frame is annotated with a zone map

$$Z(x, y) \in \{Z_{\text{NP}}, Z_{\text{dir}}^+, Z_{\text{dir}}^-, Z_{\emptyset}\}$$

where  $Z_{\text{NP}}$  denotes a no-parking zone,  $Z_{\text{dir}}^+/Z_{\text{dir}}^-$  denote zones with a defined positive/negative direction convention, and  $Z_{\emptyset}$  is unrestricted road surface.

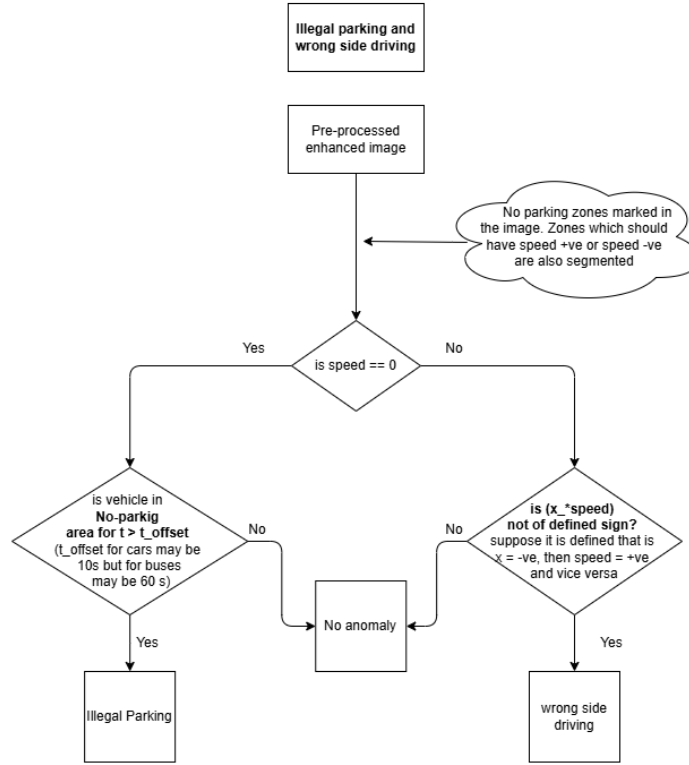


Figure 1: Illegal Parking and Wrong Side Driving

## 2.2 Motion-State Branching

For a tracked vehicle with centroid  $(x_c, y_c)$  and instantaneous speed  $v$ , the pipeline first branches on motion state:

$$\text{Branch} = \begin{cases} \text{Parking-check} & v = 0 \\ \text{Direction-check} & v \neq 0 \end{cases}$$

This single test determines which of the two independent anomaly checks is evaluated; a vehicle is never evaluated against both in the same frame.

## 2.3 Dwell-Time Gated Parking Check

For a stationary vehicle inside a no-parking zone, a per-vehicle dwell timer  $\tau_i$  accumulates the duration for which the vehicle has remained at  $v = 0$  inside  $Z_{NP}$ . The class-dependent offset  $t_{\text{offset}}(\text{class})$  absorbs brief, non-violating halts, **for example**:

$$t_{\text{offset}}(\text{class}) = \begin{cases} 10 \text{ s} & \text{class} = \text{car} \\ 60 \text{ s} & \text{class} = \text{bus} \end{cases}$$

A vehicle is flagged only once  $\tau_i$  exceeds this threshold while the vehicle remains inside the zone.

## 2.4 Sign-Mismatch Direction Check

For a moving vehicle inside a direction-governed zone, the defined convention fixes an expected sign relationship between lateral position  $x$  and speed  $v$  (e.g.,  $x < 0 \Rightarrow v > 0$ , and vice versa). The product  $x_c \cdot v$

is compared against this convention; a product whose sign does not match the defined convention for that zone indicates travel against the permitted direction.

### 3 Violation Classification Logic

The core decision logic is two independent nested conditionals, evaluated per detected vehicle, per frame:

$$\text{Illegal Parking} = \begin{cases} \text{True} & \text{if } v = 0 \wedge (x_c, y_c) \in Z_{\text{NP}} \wedge \tau_i > t_{\text{offset}}(\text{class}) \\ \text{False} & \text{otherwise} \end{cases}$$

$$\text{Wrong-Side Driving} = \begin{cases} \text{True} & \text{if } v \neq 0 \wedge \text{sign}(x_c \cdot v) \neq \text{sign}_{\text{defined}} \\ \text{False} & \text{otherwise} \end{cases}$$

#### Interpretation:

- **Illegal Parking:** the vehicle is stationary and located inside a marked no-parking zone, and has remained there beyond the class-appropriate grace period – i.e., it is not merely making a brief, legitimate stop.
- **Wrong-Side Driving:** the vehicle is in motion, and the sign of its lateral-position-speed product contradicts the direction convention defined for that road segment – i.e., it is travelling against permitted traffic flow.

## 4 Mathematical Formulation

### 4.1 Combined Decision Function

Let  $M = [v \neq 0]$  denote the motion flag,  $P = [(x_c, y_c) \in Z_{\text{NP}}]$  denote the no-parking zone membership flag,  $T = [\tau_i > t_{\text{offset}}]$  denote the dwell-time-exceeded flag, and  $W = [\text{sign}(x_c \cdot v) \neq \text{sign}_{\text{defined}}]$  denote the direction-mismatch flag. The full pipeline reduces to a single composite predicate per tracked vehicle  $i$  at frame  $t$ :

$$\text{Decision}_i^{(t)} = \begin{cases} \text{Illegal Parking} & M_i^{(t)} = 0, P_i^{(t)} = 1, T_i^{(t)} = 1 \\ \text{No anomaly} & M_i^{(t)} = 0, (P_i^{(t)} = 0 \vee T_i^{(t)} = 0) \\ \text{Wrong-Side Driving} & M_i^{(t)} = 1, W_i^{(t)} = 1 \\ \text{No anomaly} & M_i^{(t)} = 1, W_i^{(t)} = 0 \end{cases}$$

### 4.2 Boolean / Truth-Table Representation

| $M$ | $P$ | $T$ | $W$ | Outcome            |
|-----|-----|-----|-----|--------------------|
| 0   | 0   | –   | –   | No anomaly         |
| 0   | 1   | 0   | –   | No anomaly         |
| 0   | 1   | 1   | –   | Illegal Parking    |
| 1   | –   | –   | 0   | No anomaly         |
| 1   | –   | –   | 1   | Wrong-Side Driving |

This is logically equivalent to:

$$\text{Illegal Parking} = \neg M \wedge P \wedge T, \quad \text{Wrong-Side Driving} = M \wedge W$$

## 5 Algorithmic Summary

Listing 1: Per-frame illegal parking / wrong-side driving classification

```

1  T_OFFSET = {"car": 10.0, "bus": 60.0}    # seconds, class-dependent dwell
    threshold
2  EPSILON = 0.3                            # m/s, stationary threshold (tunable)
3
4  def is_stationary(speed):
5      return abs(speed) <= EPSILON
6
7  def in_no_parking_zone(cx, cy, no_parking_polygons):
8      """Point-in-polygon test against pre-segmented no-parking zones."""
9      return any(point_in_polygon(cx, cy, poly) for poly in no_parking_polygons)
10
11 def direction_mismatch(cx, speed, defined_sign):
12     """
13     defined_sign: +1 if convention is (x<0 => speed>0), -1 if reversed
14     Returns True if sign(x * speed) contradicts the defined convention.
15     """
16     if speed == 0:
17         return False
18     return (cx * speed * defined_sign) < 0
19
20 def classify_vehicle(vehicle, no_parking_polygons, defined_sign, dwell_timers):
21     """
22     vehicle: dict with keys 'id', 'cx', 'cy', 'speed', 'class'
23     dwell_timers: dict {vehicle_id: accumulated_seconds_in_zone}
24     """
25     vid, cx, cy, speed, vclass = (
26         vehicle["id"], vehicle["cx"], vehicle["cy"],
27         vehicle["speed"], vehicle["class"],
28     )
29
30     if is_stationary(speed):
31         if in_no_parking_zone(cx, cy, no_parking_polygons):
32             dwell_timers[vid] = dwell_timers.get(vid, 0.0) + FRAME_DT
33             if dwell_timers[vid] > T_OFFSET.get(vclass, 10.0):
34                 return "ILLEGAL_PARKING"
35         else:
36             dwell_timers[vid] = 0.0
37         return "NO_ANOMALY"
38     else:
39         dwell_timers[vid] = 0.0
40         if direction_mismatch(cx, speed, defined_sign):
41             return "WRONG_SIDE_DRIVING"
42         return "NO_ANOMALY"
43
44
45 def process_frame(tracked_vehicles, no_parking_polygons, defined_sign,
46                 dwell_timers):
47     """
48     tracked_vehicles: list of dicts, each with keys
49         'id', 'cx', 'cy', 'speed', 'class'
50     Returns: dict {vehicle_id: violation_label}
51     """
52     results = {}
53     for v in tracked_vehicles:
54         results[v["id"]] = classify_vehicle(

```

```

54         v, no_parking_polygons, defined_sign, dwell_timers
55     )
56     return results
57
58
59 # --- Example usage ---
60 if __name__ == "__main__":
61     FRAME_DT = 1.0 # seconds per frame (illustrative)
62     no_parking_polygons = [[(100, 400), (260, 400), (260, 520), (100, 520)]]
63     defined_sign = +1 # convention: x < 0 => speed > 0
64     dwell_timers = {}
65
66     tracked_vehicles = [
67         {"id": 1, "cx": 180, "cy": 460, "speed": 0.0, "class": "car"}, #
68         parked in zone
69         {"id": 2, "cx": -50, "cy": 300, "speed": -3.0, "class": "car"}, #
70         against convention
71         {"id": 3, "cx": 600, "cy": 300, "speed": 5.0, "class": "car"}, #
72         normal motion
73     ]
74
75     violations = process_frame(tracked_vehicles, no_parking_polygons,
76                               defined_sign, dwell_timers)
77     for vid, label in violations.items():
78         print(f"Vehicle {vid}: {label}")
79
80     # Output (after dwell exceeds threshold for vehicle 1):
81     # Vehicle 1: ILLEGAL_PARKING
82     # Vehicle 2: WRONG_SIDE_DRIVING
83     # Vehicle 3: NO_ANOMALY

```

## 6 Important Logic Points

- **No-parking and direction zones are pre-segmented, not inferred per frame.** Both zone types: no-parking regions and zones with a defined direction convention (expected positive or negative sign of motion) are marked once during the upstream segmentation stage, keeping the per-frame decision lightweight.
- **Dwell time, not mere presence, gates the parking violation.** A vehicle momentarily stationary inside a no-parking zone (e.g., passenger boarding) is not flagged; only dwell exceeding  $t_{\text{offset}}$  triggers *Illegal Parking*.
- **The offset time is class-dependent.** Buses and similar large vehicles are allowed a longer grace period (60s) than cars (10s) to account for routine stops such as boarding.
- **Direction violation is a sign-consistency check, not a magnitude check.** Only the sign of  $x_c \cdot v$  relative to the defined convention matters for *Wrong-Side Driving*; the speed magnitude plays no role once motion is confirmed.
- **Thresholding ( $\epsilon$ ) is necessary in practice** to avoid tracker/detection noise being misread as motion, since true  $v = 0$  rarely occurs exactly in pixel-derived velocity estimates.
- **Per-vehicle, per-frame evaluation.** The decision is not global to the frame; each tracked vehicle is independently evaluated against both branches' preconditions, allowing simultaneous detection of multiple, distinct violators.



# Traffic Signal Violation Detection System

## Technical Documentation & Logic Specification

*Zone-based, signal-fused detection of traffic signal violations: classifies vehicles in the detected zone during a red signal as **stop-line** or **red-light** violations.*

### 1 Overview

The **Traffic Signal Violation Detection (TSVD)** module follows a sequential decision-making pipeline to classify traffic violations based on the traffic signal state, vehicle position, and previously estimated vehicle speed. The overall flowchart is illustrated in Figure 1.

Initially, the module receives a pre-processed and enhanced CCTV frame from the image enhancement stage. The image is then logically divided into two regions: a *Normal Zone*, representing the area before the stop line, and a *Detection Zone*, representing the area beyond the stop line where violations are evaluated.

Unlike conventional vision-based systems, the current traffic signal state is not inferred from the CCTV image but is directly obtained from the traffic control system. This eliminates errors caused by poor illumination, occlusions, adverse weather conditions, and camera perspective.

The first decision block checks whether the current signal state is *Red*. If the signal is Green or Yellow, every detected vehicle is immediately classified as *No Anomaly*, and no further processing is required.

If the signal is Red, the module verifies whether the detected vehicle lies inside the predefined Detection Zone. Vehicles remaining outside this region are considered compliant with traffic regulations and are therefore classified as *No Anomaly*.

For vehicles located inside the Detection Zone, an anomaly is identified and the previously computed vehicle speed is evaluated. If the vehicle speed is greater than zero, the system classifies the event as a *Red-Light Violation*, indicating that the vehicle has crossed the stop line while continuing to move during a red signal. Conversely, if the vehicle speed is equal to zero, the event is classified as a *Stop-line Violation*, indicating that the vehicle has crossed the stop line and stopped outside the marked area.

Thus, the proposed pipeline performs traffic violation detection through a deterministic sequence of logical decisions without requiring additional image processing or deep learning inference. Since each vehicle undergoes only a constant number of comparisons, the overall *computational complexity remains linear with respect to the number of detected vehicles*, making the system suitable for real-time intelligent traffic monitoring applications.

### 2 Stage-by-Stage Description

The pipeline takes the **pre-processed, enhanced frame**  $I_{enh}$  as its input (image acquisition and enhancement are assumed already complete upstream).

#### 2.1 Spatial Zoning (Normal vs. Detected Zone)

The enhanced frame is partitioned into two disjoint regions using the stop-line position as the separating boundary:

$$I_{enh} = Z_N \cup Z_D, \quad Z_N \cap Z_D = \emptyset$$

where  $Z_N$  is the *Normal Zone* (before the stop line, legal waiting area) and  $Z_D$  is the *Detected Zone* (at/beyond the stop line, into the intersection).

A vehicle, represented by its bounding-box centroid  $(x_c, y_c)$ , is determined to be in the detected zone using a point-in-polygon / half-plane test. For a stop-line modeled as a line  $ax + by + c = 0$ :

$$\text{Zone}(x_c, y_c) = \begin{cases} Z_D & \text{if } ax_c + by_c + c > 0 \\ Z_N & \text{if } ax_c + by_c + c \leq 0 \end{cases}$$

#### 2.2 Signal-State Fusion

Unlike the spatial zoning, the traffic signal color  $S$  is **not inferred visually** from the camera frame; it is supplied directly by the control room as ground truth:

$$S \in \{\text{Red, Yellow, Green}\}$$

This avoids ambiguity/errors from signal-light recognition under glare, occlusion, or low-light conditions, and keeps the violation decision logic deterministic and auditable.

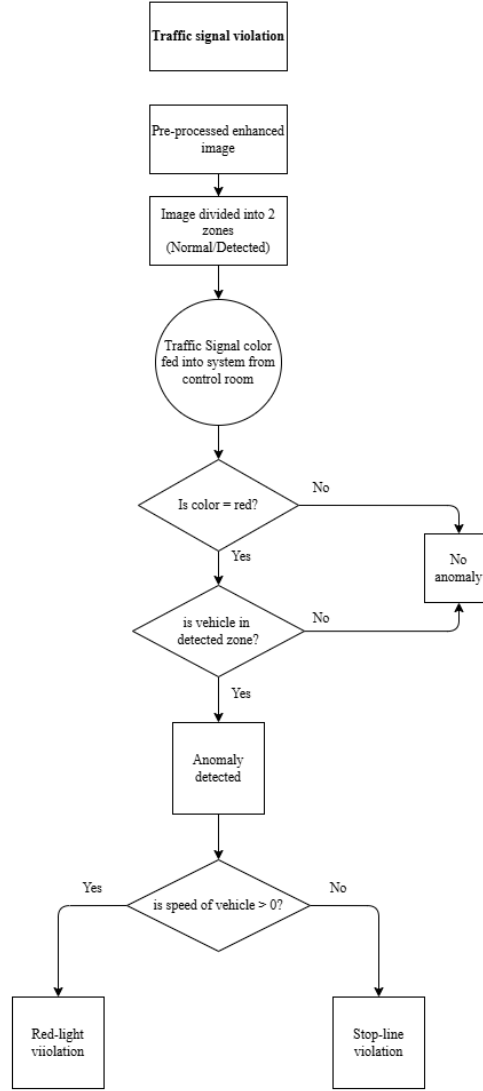


Figure 1: Traffic Signal Violation Flowchart

### 2.3 Violation Classification Logic

The core decision logic is a nested conditional evaluated per detected vehicle, per frame:

$$\text{Anomaly} = \begin{cases} \text{True} & \text{if } S = \text{Red} \wedge \text{Zone}(x_c, y_c) = Z_D \\ \text{False} & \text{otherwise} \end{cases}$$

When an anomaly is flagged, the vehicle's instantaneous speed  $v$  disambiguates the violation type:

$$\text{Violation Type} = \begin{cases} \text{Stop-line violation} & \text{if } v = 0 \\ \text{Red-light violation} & \text{if } v > 0 \end{cases}$$

#### Interpretation:

- **Stop-line violation:** the vehicle has advanced into the detected (restricted) zone but is currently stationary – e.g., it crept over the stop line and halted.
- **Red-light violation:** the vehicle is in the detected zone *and* still moving while the signal is red – an active light-running event.

### 3 Mathematical Formulation

#### 3.1 Combined Decision Function

Vehicle speed  $v$  (and hence motion state) is assumed already computed upstream by the tracking module; let  $M = [v > \epsilon]$  denote the moving flag. The full pipeline reduces to a single composite predicate per tracked vehicle  $i$  at frame  $t$ :

$$\text{Decision}_i^{(t)} = \begin{cases} \text{No anomaly} & S^{(t)} \neq \text{Red} \\ \text{No anomaly} & S^{(t)} = \text{Red}, \text{Zone}_i^{(t)} = Z_N \\ \text{Stop-line violation} & S^{(t)} = \text{Red}, \text{Zone}_i^{(t)} = Z_D, M_i^{(t)} = 0 \\ \text{Red-light violation} & S^{(t)} = \text{Red}, \text{Zone}_i^{(t)} = Z_D, M_i^{(t)} = 1 \end{cases}$$

#### 3.2 Boolean / Truth-Table Representation

Let  $R = [S = \text{Red}]$ ,  $D = [\text{Zone} = Z_D]$  (Iverson brackets). The classification reduces to:

| $R$ | $D$ | $M$ | Outcome             |
|-----|-----|-----|---------------------|
| 0   | –   | –   | No anomaly          |
| 1   | 0   | –   | No anomaly          |
| 1   | 1   | 0   | Stop-line violation |
| 1   | 1   | 1   | Red-light violation |

This is logically equivalent to:

$$\text{Violation} = R \wedge D, \quad \text{Type} = \begin{cases} \text{Stop-line} & \text{if } \neg M \\ \text{Red-light} & \text{if } M \end{cases} \quad (\text{given Violation} = 1)$$

### 4 Algorithmic Summary

Listing 1: Per-frame violation classification

```

1 EPSILON = 0.3 # m/s, motion threshold (tunable)
2
3 def get_zone(cx, cy, a, b, c):
4     """Half-plane test against the stop-line: ax + by + c = 0"""
5     return "DETECTED" if (a * cx + b * cy + c) > 0 else "NORMAL"
6
7 def classify_violation(signal_color, cx, cy, speed, stop_line):
8     """
9     signal_color : str -> 'RED' | 'YELLOW' | 'GREEN' (from control room)
10    cx, cy       : float -> vehicle centroid (pixels)
11    speed        : float -> precomputed vehicle speed (m/s)
12    stop_line    : tuple -> (a, b, c) line coefficients
13    """
14    if signal_color != "RED":
15        return "NO_ANOMALY"
16
17    zone = get_zone(cx, cy, *stop_line)
18    if zone == "NORMAL":
19        return "NO_ANOMALY"
20
21    # Anomaly detected: vehicle is in the detected zone on red
22    if speed <= EPSILON:
23        return "STOP_LINE_VIOLATION"
24    else:
25        return "RED_LIGHT_VIOLATION"
26
27
28 def process_frame(signal_color, tracked_vehicles, stop_line):
29     """
30     tracked_vehicles : list of dicts, each with keys
31                       'id', 'cx', 'cy', 'speed'
32     Returns: dict {vehicle_id: violation_label}
33     """

```

```

34     results = {}
35     for v in tracked_vehicles:
36         results[v["id"]] = classify_violation(
37             signal_color, v["cx"], v["cy"], v["speed"], stop_line
38         )
39     return results
40
41
42 # --- Example usage ---
43 if __name__ == "__main__":
44     stop_line = (0, 1, -480)           # e.g. horizontal line y = 480
45     signal_color = "RED"
46     tracked_vehicles = [
47         {"id": 1, "cx": 320, "cy": 510, "speed": 0.0},    # past line, stopped
48         {"id": 2, "cx": 340, "cy": 540, "speed": 4.2},    # past line, moving
49         {"id": 3, "cx": 300, "cy": 400, "speed": 6.0},    # before line
50     ]
51
52     violations = process_frame(signal_color, tracked_vehicles, stop_line)
53     for vid, label in violations.items():
54         print(f"Vehicle {vid}: {label}")
55
56     # Output:
57     # Vehicle 1: STOP_LINE_VIOLATION
58     # Vehicle 2: RED_LIGHT_VIOLATION
59     # Vehicle 3: NO_ANOMALY

```

## 5 Important Logic Points

- **Signal color is externally supplied, not vision-inferred.** The system trusts the control room feed for  $S$ , eliminating signal-light-recognition failure modes (glare, occlusion, faded bulbs) as a source of false negatives/positives.
- **Red is the only gating condition.** No check is performed against Green or Yellow states – any non-red signal short-circuits the pipeline to “No anomaly” regardless of vehicle position.
- **Zone test acts as a spatial gate.** Only vehicles whose centroid crosses into  $Z_D$  while  $S = \text{Red}$  are escalated; vehicles waiting correctly in  $Z_N$  are never flagged.
- **Speed is the sole discriminator between the two violation subtypes**, not an independent trigger – it only matters *after* an anomaly is already confirmed.
- **$v = 0$  does not imply “no violation.”** A stationary vehicle that has already crossed the stop line during red is still a violation (Stop-line violation), just a different category from an actively moving one.
- **Thresholding ( $\epsilon$ ) is necessary in practice** to avoid tracker/detection noise being misread as motion, since true  $v = 0$  rarely occurs exactly in pixel-derived velocity estimates.
- **Per-vehicle, per-frame evaluation.** The decision is not global to the frame; each tracked vehicle is independently evaluated, allowing simultaneous detection of multiple violators.

# Cross-Camera Evidence Correlation for Flagged Vehicles

## Using Spatio-Temporal Reasoning over a Camera Topology Graph

### 1 Purpose and Scope

The existing single-camera pipeline detects a range of traffic violations independently at each camera site: overspeeding, helmet and seatbelt non-compliance, triple riding, stop-line and red-light violations, illegal parking, and wrong-side driving. Each of these detections is generated, stored, and reported as an isolated event tied to one camera at one point in time. This document describes the design of a complementary system that sits above the existing per-camera pipeline. Its purpose is to determine, once a vehicle has been flagged by any single camera, whether the same vehicle is later observed by other cameras in the network, so that all evidence relating to that vehicle across the entire camera network can be consolidated into a single case file rather than remaining scattered across independent camera logs.

The central idea is that a flagged vehicle's future position in the camera network can be predicted from the known physical layout of the road network and historical traffic behaviour, without requiring an expensive or unreliable city-wide visual search. Once a likely time window and a likely set of downstream cameras have been identified, a lightweight confirmation step (number-plate matching, with appearance matching as a fallback) is used to verify that the vehicle observed at a downstream camera is in fact the same vehicle that was originally flagged.

### 2 Camera Topology Graph

#### 2.1 Structure

The camera network is represented as a graph whose nodes are the physical camera installations and whose edges represent direct, traversable road connections between two cameras. An edge is drawn between two cameras only if a vehicle can travel from one to the other without passing through a third monitored camera in between. Each edge carries a weight that represents the expected travel time, in seconds, for a vehicle travelling along that specific road segment between the two cameras.

In the reference implementation, three cameras, denoted Cam1, Cam2, and Cam3, are connected as follows: an edge of weight 40 connects Cam1 and Cam2, an edge of weight 60 connects Cam2 and Cam3, and an edge of weight 50 connects Cam1 and Cam3 directly. The presence of a direct edge between Cam1 and Cam3, in addition to the path through Cam2, reflects a real situation in which more than one physical road allows a vehicle to travel between two points, and each such road is represented as its own edge rather than being collapsed into a single shortest-path value. This is important because a vehicle leaving Cam1 may legitimately reach Cam3 by either route, and the system must be able to search for it at the times implied by both possibilities rather than only the faster one.

The edge weights are not fixed physical constants such as distance divided by speed limit. They

are derived from a model trained on historical transit-time distributions for each camera pair, incorporating the road distance between the two cameras as well as the number of intersections, signals, or other points of interruption that lie on that specific stretch of road. Two camera pairs that are the same straight-line distance apart can have different edge weights if one route passes through more signal-controlled junctions than the other, since each such junction introduces additional, variable delay. The edge weight stored in the graph should therefore be understood as a mean transit time,  $\mu$ , with an associated spread,  $\sigma$ , learned from past traffic rather than from a single theoretical calculation.

## 2.2 Role of the Number-Plate Detector Camera

In addition to the cameras that participate in the general topology graph, the network includes a designated number-plate detector camera, referred to here as CamX to avoid confusion with the graph node already named Cam1 in the reference diagram. CamX is the camera at which the system attempts to read the number plate of a candidate vehicle once a time search window for that vehicle has been computed. CamX may correspond physically to any node in the topology graph; it is distinguished in this document purely by its functional role as the point at which plate confirmation is attempted, not by a separate physical location.

## 3 Shared State

All cameras in the network write to and read from a common shared state store. The shared state holds, for each actively tracked vehicle, a record analogous to the one already produced by the existing per-camera pipeline: a vehicle identifier, the set of violation flags raised so far, the camera and timestamp at which each flag was raised, and, once available, the vehicle's number plate. The shared state is the mechanism by which a flag raised at one camera becomes visible to the logic that computes time search windows for other cameras, and by which a confirmed plate match at a downstream camera is written back so that the case file continues to accumulate evidence rather than being treated as a new, unrelated event.

## 4 End-to-End Pipeline

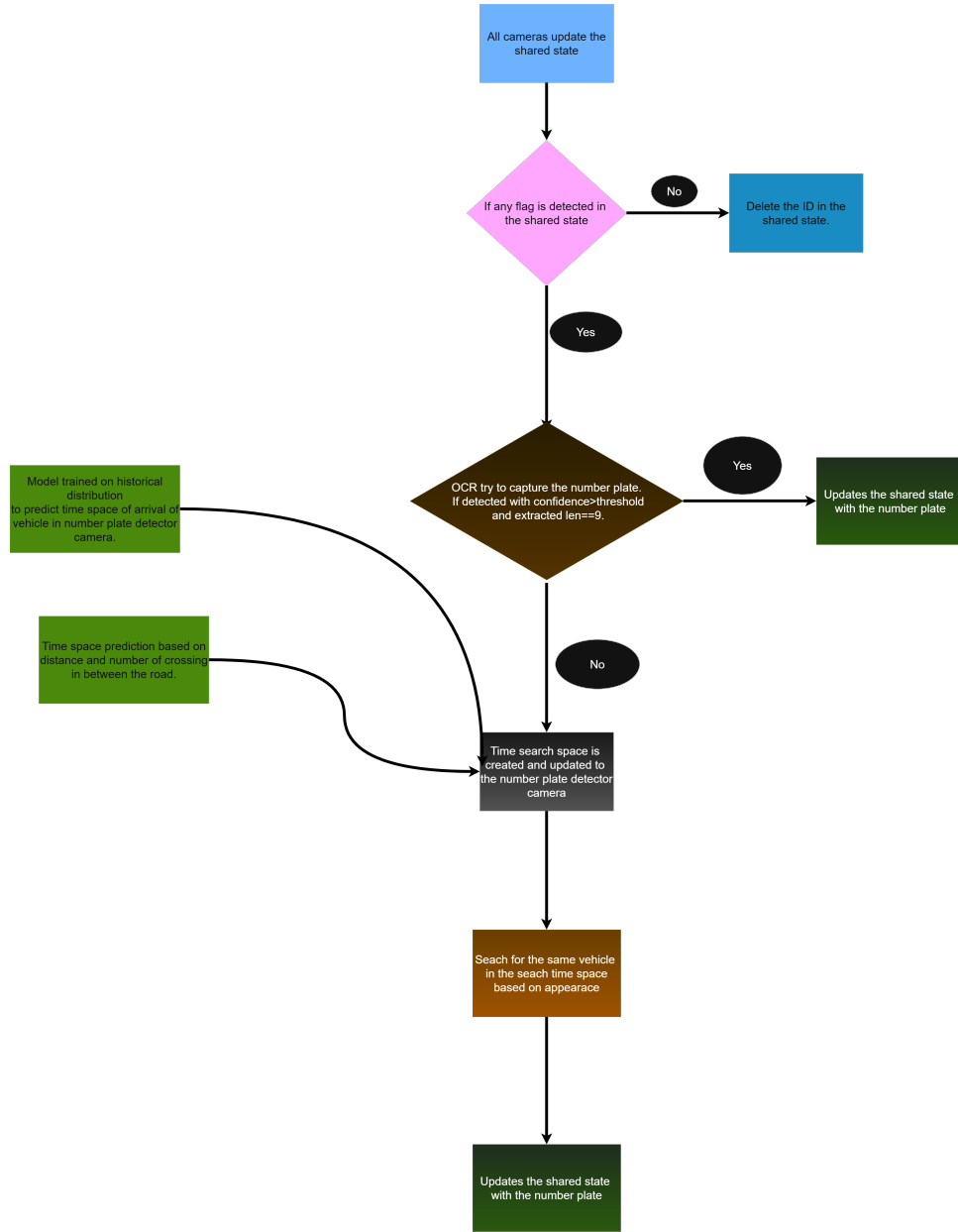


Figure 1: Workflow of evidence extractor.

The pipeline begins at the point where the existing per-camera violation detectors have already run. From there, the following sequence is followed.

### 4.1 Step 1: Shared State Update

Every camera, after completing its own local violation detection, writes its findings into the shared state. This is a continuous process and is not contingent on a violation having occurred; a vehicle with no violations is also written so that, if it is flagged later by a different camera further down its route, its earlier appearance is already available.

## 4.2 Step 2: Flag Check

The shared state is checked for the presence of any violation flag attached to a given vehicle identifier. If no flag is present, the vehicle’s entry is deleted from the shared state. This deletion step prevents the shared state from growing indefinitely with vehicles that present no evidentiary interest, since the purpose of the downstream pipeline is specifically to build a cross-camera case file for vehicles that have already been flagged for at least one violation. If a flag is present, the vehicle proceeds to the next step.

## 4.3 Step 3: Time Search Space Construction

For a flagged vehicle, the system constructs a time search space describing when and where the vehicle is expected to next appear in the camera network. This is done in two parts.

First, the distance-based component of the prediction is computed using the camera topology graph described in Section 2: the road distance between the camera that raised the flag and each reachable downstream camera, together with the number of intersections lying on that road segment, is used to estimate a baseline transit time for each candidate edge.

Second, this baseline is passed through a model trained on the historical transit-time distribution for that specific camera pair, which converts the baseline estimate into an expected arrival window rather than a single arrival instant. The result is, for each reachable downstream camera, a time interval during which the flagged vehicle is expected to arrive, centred on the historically observed mean transit time and widened according to the historically observed variance for that route. This interval is what is referred to as the time search space, and it is this interval, rather than a single predicted timestamp, that is written and updated against the number-plate detector camera, CamX, that lies on the relevant route.

## 4.4 Step 4: Number-Plate Confirmation at the Search Window

When CamX’s own clock falls within the time search space written for a given vehicle, the system attempts to read the number plate of vehicles passing CamX during that window using the existing OCR component of the pipeline. A read is accepted only if the OCR confidence exceeds a defined threshold and the extracted string has the expected length of nine characters, corresponding to a standard number-plate format. If a plate is successfully read and accepted, it is written into the shared state against the vehicle identifier, linking the new sighting at CamX to the original flagged record.

If no plate is successfully read and accepted within the time search space, the system does not immediately discard the candidate. Instead, it falls back to an appearance-based search: the candidates observed at CamX within the time search space are compared against the appearance of the originally flagged vehicle, and the same shared-state update is performed if an appearance match is found. This fallback exists specifically to handle the cases in which plate OCR fails due to poor viewing angle, occlusion, low light, or a damaged or obscured plate, which are common in practice and would otherwise cause a genuine match to be lost.



## 4.5 Step 5: Consolidation

Once the shared state for a vehicle identifier contains a confirmed plate (or appearance) match at a downstream camera, that camera’s evidence is merged into the same case file as the originally flagged violation. Over time, as a flagged vehicle continues to move through the network, this process repeats at each subsequent camera reachable from its current position, so that the final case file accumulates every relevant piece of evidence gathered across the network, rather than each camera’s findings remaining isolated.

## 5 Worked Example of the Time Search Space

The following example illustrates why the time search space is expressed as a window rather than a single predicted time, using the edge between two cameras two kilometres apart with a posted speed limit of forty kilometres per hour.

Historical logging of ordinary, non-flagged traffic along this specific route, collected over several weeks by the existing tracking and OCR pipeline, yields a mean transit time of four minutes with a standard deviation of one minute. This already establishes that even unflagged, typical traffic does not take a fixed time to cover this distance; some vehicles are delayed by a signal along the way, while others pass through more quickly.

A naive point-estimate approach would compute distance divided by speed limit, giving three minutes, and would check the downstream camera’s footage only at the single instant three minutes after the flag was raised. This approach fails in practice for two reasons: a vehicle delayed by a signal may arrive several minutes later than this estimate, and a vehicle that was itself flagged for speeding may arrive earlier than the speed-limit-based estimate would suggest, since it is, by definition, not travelling at the legal limit.

The window-based approach instead uses the historical mean and standard deviation directly. Taking a window of mean plus or minus two standard deviations around a flag raised at fourteen hundred thirty hours produces a search interval from fourteen-thirty-two to fourteen-thirty-six. Searching the downstream camera’s records across this four-minute interval, rather than at a single instant, captures the vehicle whether it travelled faster, slower, or was delayed by a signal, while still keeping the search space small enough to be practical. The choice of how many standard deviations to include controls a direct trade-off: a wider window increases the chance of catching the correct vehicle but also increases the number of irrelevant vehicles that must be filtered out by the plate or appearance confirmation step, while a narrower window is cleaner but risks missing the vehicle entirely on days when traffic conditions deviated from the historical norm.

## 6 Why Confirmation Is Required in Addition to Timing

The time search space narrows the set of vehicles that must be checked at a downstream camera, but it does not, by itself, identify which vehicle within that window is the one originally flagged, since other unrelated vehicles will ordinarily also pass through the same camera during the same interval. For this reason, the pipeline always pairs the time search space with a confirmation step. Number-plate matching is used as the primary confirmation signal because the existing pipeline already performs plate OCR as part of its standard evidence-gathering process, and a plate match is a strong, low-ambiguity identifier. Appearance-based matching is retained as a fallback specifically for the cases in which OCR confidence or extracted plate length fails to meet the required threshold, ensuring that a genuine match is not lost solely because of an imperfect plate read.

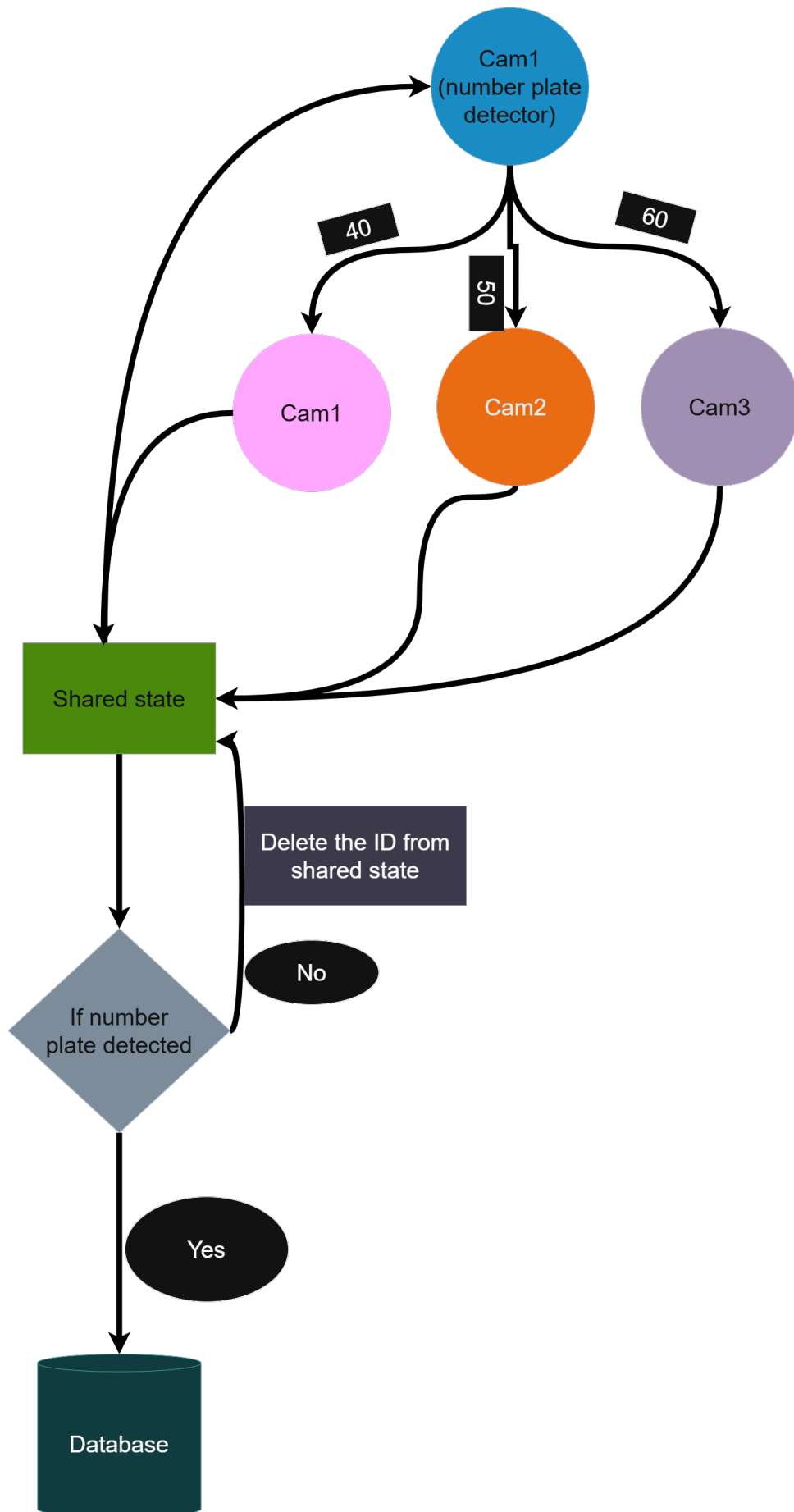


Figure 2: graph of evidence extractor.

## 7 Summary

The system extends the existing per-camera violation pipeline with a network-level layer that links a flagged vehicle’s evidence across multiple cameras. This is achieved by representing the physical camera network as a topology graph with empirically derived, time-distributed edge weights; using that graph, together with a flag raised in the shared state, to compute a time search space at each reachable downstream camera rather than a single predicted arrival time; and confirming candidate matches within that search space using number-plate OCR as the primary signal and appearance similarity as a fallback. The result is a single, consolidated case file per flagged vehicle that draws together every violation and every camera sighting relevant to that vehicle across the network, rather than a set of disconnected, single-camera records.